

Programa Institucional de Bolsas de Iniciação Científica – PIBIC
RELATÓRIO FINAL

IDENTIFICAÇÃO DO PROJETO

Título do Projeto: Aplicando estratégias de diversificação em algoritmos evolutivos para problemas de Otimização em Engenharia Elétrica			
Local de Realização Unidade/Instituto/Departamento/Laboratório): Escola de Engenharia / Departamento de Engenharia Elétrica			
Endereço: Rua Passo da Pátria 156, Sala 421 Bloco D			
Bairro: São Domingos	Cidade: Niterói	UF: RJ	CEP: 24210-240

DADOS DO ORIENTADOR

Nome: Rainer Zanghi			
Siape: 2393115		CPF: 03420509740	
Endereço: Rua Passo da Pátria 156, Sala 421 Bloco D			
Bairro: São Domingos		Cidade: Niterói	
UF: RJ	CEP: 24210-240	E-mail: rzanghi@id.uff.br	
Telefone 1: (21) 2629-5233		Telefone 2: (____) _____	

DADOS DO BOLSISTA

Nome: Pedro Victor Rodrigues Veras			
Matrícula: 622038083		CPF: 13206502799	CR: 6,0
Curso/Departamento/Instituto: Engenharia Elétrica/TEE/Universidade Federal Fluminense			
Endereço: Rua da conceição, 101			
Bairro: Centro		Cidade: Niterói	
UF: RJ	CEP: 24020-085	E-mail: pedrovictorveras@id.uff.br	
Telefone 1: (21) 999289987		Telefone 2: (____) _____	

SUMÁRIO

- 1) Título
- 2) Introdução
- 3) Metodologia
 - 3.1) Levantamento Bibliográfico e Estudo do Caso de Agendamento
 - 3.2) Consolidação dos Conceitos e Elaboração do Algoritmo
 - 3.3) Implementação e Simulação
 - 3.4) Arquitetura e ferramentas
 - 3.5) Exemplo de implementação de uma função objetivo para um SEP
 - 3.6) Caso de uso para o problema de agendamento ótimo da rede IEEE 14
- 4) Produção técnico-científica
- 5) Resultados
 - 5.1) Simulações do problema de agendamento ótimo na rede IEEE 14
 - 5.2) Simulações do problema de agendamento ótimo na rede IEEE 30
 - 5.3) Simulações do problema de agendamento ótimo na rede IEEE 118
 - 5.4) Simulações no SIN 45 como exemplo de controle de casos
 - 5.5) Passo a passo para utilização do *Framework* pelo *Github*
- 6) Conclusões
- 7) Referências bibliográficas
- 8) Auto-avaliação do aluno

1) TÍTULO: Aplicando estratégias de diversificação em algoritmos evolutivos para problemas de Otimização em Engenharia Elétrica

2) INTRODUÇÃO

A operação dos sistemas elétricos de potência (SEP) tornou-se uma tarefa complexa, impulsionada pela necessidade de equilibrar custos e segurança em um cenário cada vez mais exigente. A eficiência e a confiabilidade desses sistemas dependem de análises computacionais, especialmente em um contexto onde a otimização tradicional enfrenta desafios devido à falta de correlação direta entre critérios de avaliação e variáveis de decisão.

As metaheurísticas são estratégias em *software* para realizar uma busca eficiente do espaço de soluções para problemas de otimização que não podem ser abordados pela Otimização Matemática. Dentre estas estratégias, os algoritmos evolutivos são ferramentas inspiradas em processos biológicos, como a seleção natural. Essas técnicas, como apresentadas para o agendamento de intervenções em redes elétricas [1, 2], permitem uma exploração eficiente e adaptativa do espaço de soluções, essencial para enfrentar os desafios dos SEP modernos [3]. A complexidade dos SEP atuais, marcada pela diversidade de fontes energéticas e tecnologias de transporte, exige análises profundas, repetitivas e exaustivas. Essas análises, muitas vezes de alto custo computacional, são um desafio para sistemas de grande porte, destacando a necessidade de inovação em ferramentas de otimização e análise [3].

O desenvolvimento de *frameworks* de *software* auxilia a pesquisa em análise de SEP, especialmente quando incorporam facilidades de implementação e utilizam bibliotecas de código aberto. A linguagem *Python*, por sua acessibilidade e facilidade de aprendizado, torna-se uma boa ferramenta para essas tarefas.

O *Pandapower* [4] se destaca como uma biblioteca de código aberto que automatiza análises de sistemas elétricos de transmissão e distribuição. Sua eficiência na representação de redes elétricas é comparável com *softwares* comerciais, tornando-o uma ferramenta para estudos avançados em *smart grids* [5], além de ser uma ferramenta que permite simulações do comportamento de redes elétricas.

As *smart grids* [5] representam uma revolução no conceito tradicional de redes elétricas, incorporando tecnologias avançadas de comunicação e controle, além de uma diversidade de fontes energéticas. Essa nova concepção amplia o fluxo de energia e informações, essencial para a eficiência e a resiliência dos sistemas elétricos modernos. As *Smart Grids* combinam tecnologia da informação e comunicação com a infraestrutura elétrica para uma gestão mais eficiente da energia, na presença de diversas formas de gestão energética. Um exemplo é a Gestão Ativa de Demanda, onde são propostos modelos que permitem aos consumidores ajustar seu consumo em tempo real para reduzir custos e aliviar a carga na rede. Outro tema de importância nas *smart grids* é a Segurança Cibernética, onde é possível desenvolver protocolos e arquiteturas de segurança para proteger as redes inteligentes contra ataques cibernéticos. Complementando os exemplos, pode-se citar as Microgrids com

Geração Distribuída que são redes elétricas locais que podem operar de forma autônoma, onde podem ser feitas análises, planejamento e operação otimizada. Nas *smart grids*, a Internet das Coisas (*IoT*) e a automação permitem a conexão de dispositivos físicos a sistemas digitais, possibilitando a criação de redes que se comunicam de forma inteligente. Assim como é utilizado na Automação Industrial focada em eficiência energética, é possível desenvolver sistemas de automação que utilizam sensores e IA para otimizar o consumo de energia em edificações. É possível projetar sistemas baseados em *IoT* para o monitoramento em tempo real e controle de infraestruturas elétricas e de gerenciamento de tráfego de Cidades Inteligentes.

No contexto das *smart grids*, o *Pandapower* [4] é um *framework* de código aberto projetado para modelar, analisar e otimizar sistemas de potência. A natureza colaborativa e *open source* do *Pandapower* acelera o desenvolvimento de software, permitindo que a comunidade contribua com melhorias e adaptações. Além disso, a capacidade do *Pandapower* de integrar-se com outras bibliotecas *Python* de alto desempenho, muitas delas implementadas em linguagens compiladas como *Fortran* e *C/C++*, viabiliza a transição de protótipos acadêmicos para soluções comerciais mais eficientes. Isso significa que as inovações desenvolvidas no ambiente acadêmico podem ser facilmente escaladas para aplicação prática, beneficiando diretamente a operação das redes elétricas inteligentes.

O problema do agendamento ótimo de intervenções em redes elétricas foi abordado em [1], onde foram propostas diversas técnicas para sua resolução de forma eficiente. Esta abordagem é importante para mitigar o alto custo de avaliação da sua função objetivo, que executa algoritmos de fluxo de potência para verificar violações em diversos cenários. Este problema tem algumas características que tornam atraente o uso de metaheurísticas para sua solução, como a interdependência entre os valores das variáveis de decisão e uma relação não equacionável entre as variáveis de decisão e o resultado da função objetivo.

Uma estratégia comum aplicada em problemas de otimização discreta consiste em computar e armazenar valores parciais de aptidão [2], associados à ocorrência de genes específicos, que são resgatados e usados sempre que aquela sequência dos genes. No entanto, esta estratégia não pode ser aplicada em problemas epistáticos, em que um único gene não pode ser diretamente associado ao aumento ou diminuição do valor de aptidão. Nesta classe de problemas, devido às relações de dependência entre os genes, a função de aptidão deve ser completamente avaliada para cada indivíduo único. De modo a reduzir o custo computacional destes algoritmos, é possível construir uma lista com todas as soluções distintas já exploradas pelo processo de busca [1] e evitar que o algoritmo seja executado diversas vezes para avaliar a função de aptidão do mesmo indivíduo.

Em um AE, no entanto, diversificação resulta em um aumento do custo computacional total, tendo em vista que mais avaliações da função de aptidão são necessárias quando cresce o número de indivíduos distintos entre si. Alternativamente, para reduzir o custo computacional, pode-se encurtar a duração do estágio de diversificação do processo evolutivo (possivelmente com perda de qualidade da solução). Deve-se notar a dificuldade em lidar com a relação de compromisso entre a manutenção da diversificação para melhoria da

qualidade da solução e os prejuízos trazidos pelo consequente aumento do esforço computacional em explorar áreas que não proporcionam melhoria de qualidade.

O agendamento de intervenções em redes elétricas é uma tarefa importante no planejamento da operação de SEP. Esta análise requer uma avaliação das solicitações de intervenção para garantir que a segurança operacional não seja comprometida [2]. Neste contexto, a responsabilidade recai sobre o operador, analistas e técnicos, que devem avaliar as solicitações de intervenção, ponderando os impactos sistêmicos e as restrições operativas. Por esse motivo, o planejamento da operação de redes elétricas exige uma alta confiabilidade na modelagem matemática do sistema simulado.

Nos trabalhos [1], [2], a representação dos agendamentos no algoritmo evolutivo é feita através de uma codificação inteira dos genótipos. Cada cromossomo representa um conjunto de genes, onde cada gene corresponde a um horário inicial do desligamento a ser programado. Com uso da biblioteca *DEAP*, o *framework* feito em [6] pode ser modificado para realizar diferentes tipos de configurações de AG usando sua própria função objetivo.

A segurança operacional é um fator que deve ser avaliado considerando diferentes tipos de cenários que compõem cada programação de desligamento. Esses cenários incluem variações nas condições de geração de energia, demanda de carga e topologia da rede [1]. A função de aptidão orienta o processo evolutivo, buscando minimizar as violações da segurança operacional do sistema elétrico durante o período das manutenções, considerando todas as violações de tensão nas barras e fluxo nos ramos como linhas e transformadores.

Este projeto PIBIC dá continuidade ao projeto anterior “IC230299 - ESTRATÉGIAS DE DIVERSIFICAÇÃO EM META-HEURÍSTICAS APLICADAS A PROBLEMAS DE OTIMIZAÇÃO EM ENGENHARIA ELÉTRICA” [6] onde foi desenvolvido um *framework open source* usando a biblioteca *DEAP* focada em algoritmos evolutivos para resolver problemas envolvendo metaheurísticas e aplicando a estratégia RCE (Repopulação Conjunto Elite) [7]. O objetivo principal do atual trabalho é desenvolver e validar uma metodologia para o agendamento de intervenções em redes elétricas que combine a simulação de redes elétricas com *Pandapower* e a otimização proporcionada por algoritmos evolutivos do *framework* em *DEAP* desenvolvido anteriormente. Dessa forma, a metodologia será aplicada a um caso de uso de planejamento da operação de sistemas elétricos, buscando um agendamento que minimize as violações operacionais e maximize a segurança do sistema. Também é apresentado o desenvolvimento de *frontend* para visualização das simulações.

A eficácia da metodologia proposta foi inicialmente avaliada por meio de simulações realizadas em sistemas modelo do IEEE, incluindo os sistemas de 14, 30 e 118 barras e SIN 45, onde foi calculada a função objetivo de um agendamento qualquer nestes sistemas, considerando a metodologia proposta em [1].

3) METODOLOGIA

3.1) Levantamento Bibliográfico e Estudo do Caso de Agendamento

Inicialmente, realizou-se um levantamento bibliográfico abrangente sobre o tema de agendamento de manutenção em redes elétricas, com foco na utilização de *Pandapower*. A documentação do *Pandapower* [4] foi estudada junto com vídeos tutoriais feitos pelos colaboradores para compreender as funcionalidades e recursos disponíveis para a modelagem e simulação de redes elétricas, bem como para a implementação do algoritmo de fluxo de potência. O estudo do caso de agendamento se baseou na identificação de um problema real de planejamento da operação de sistemas elétricos, onde a necessidade de agendar manutenções de forma otimizada se mostrou evidente. Para este fim foram estudadas as referências [1, 2 e 7]. O bolsista se propôs a participar de diversos workshops de cursos online como Alura e FIAP para estudos da linguagem *Python* para análise de dados com foco nas bibliotecas *Pandas* e *Streamlit*.

3.2) Consolidação dos Conceitos e Elaboração do Algoritmo

A segunda etapa consistiu na consolidação dos conceitos fundamentais para a implementação do algoritmo de agendamento proposto em [1]. Foram definidos os principais elementos da rede elétrica a serem considerados, como barras, linhas, transformadores e geradores, bem como as variáveis relevantes para a análise, como potência ativa, reativa e aparente, e tensão nas barras. Conceitos como fluxo de potência, análise de contingências e diferentes cenários operacionais foram aprendidos com base na literatura e explicações com o professor e adaptados para o contexto do problema de agendamento. O algoritmo foi estruturado para considerar diferentes cenários de operação da rede, incluindo contingências (falhas em componentes) e diferentes programações de desligamento para manutenção. O objetivo principal do algoritmo que calcula a função objetivo é determinar as violações operacionais na rede, como sobrecarga de equipamentos ou violação de limites de tensão, para um determinado agendamento. O algoritmo de otimização utilizará esta função para avaliar cada agendamento explorando diversos cenários no espaço de busca.

Em sistemas que operem interligados, tal como no Brasil o Sistema Interligado Nacional (SIN), diversos agentes programam desligamentos para manutenção de seus equipamentos e encaminham requisições de tais intervenções ao operador do sistema. Cabe então ao operador a tarefa de conciliar todas as requisições solicitadas, de modo a que seja mantida a operação segura do sistema. Diferentes restrições operativas devem ser avaliadas de modo a levar a uma programação que atenda, da melhor forma possível [1]. O agendamento de intervenções será otimizado através do cálculo do melhor cenário, garantindo a segurança e a estabilidade das redes. O algoritmo deverá ser capaz de lidar com restrições operacionais e regulatórias, garantindo que as intervenções sejam realizadas de forma eficiente e em conformidade com as normas técnicas. Assim como em [2], neste trabalho foram consideradas as restrições de subtensão e sobretensão das barras, atendimento a

demanda (convergência do fluxo de potência) e limite de fluxo de potência aparente nos ramos. Também foi considerada uma lista de contingências simples para cada cenário de interesse e três patamares de carga em uma janela horária de 32 horas. Todos os critérios podem ser ajustados no algoritmo para ampliar a pesquisa para novas abordagens do problema.

3.3) Implementação e Simulação

A terceira etapa envolveu a implementação do algoritmo de agendamento utilizando *Pandapower* e *Python*. Foi desenvolvida uma classe em *Python* como interface para a modelagem da análise de uma rede elétrica, permitindo a configuração dos dados de entrada, como topologia da rede e parâmetros dos equipamentos. O algoritmo de agendamento foi implementado dentro desta classe, utilizando as funcionalidades do *Pandapower* para realizar simulações de fluxo de potência, considerando perfis de carregamento, desligamentos e contingências. Para demonstrar o funcionamento do algoritmo, foram criados cenários de simulação com diferentes programações de desligamento para manutenção, conforme presentes em [1]. Um fluxograma simples foi elaborado para visualizar o fluxo do algoritmo, e trechos de código *Python* foram implementados de forma clara e concisa para facilitar a compreensão e a replicabilidade da metodologia. O foco na implementação foi a simplicidade, buscando um código didático e de fácil entendimento.

Foi disponibilizado ao final do projeto uma ferramenta de otimização em código aberto para aplicações dos SEP com uma interface gráfica, considerando o uso de bibliotecas existentes e de fácil acesso. O projeto possui um repositório online e colaborativo chamado de *Repopulation-With-Elite-Set (RCE)* que pode ser encontrado em:

<https://github.com/PedroVic12/Repopulation-With-Elite-Set>

A aplicação implementada consiste em uma solução de *software* de código aberto projetada para a otimização do planejamento e agendamento de intervenções (manutenções) em redes elétricas de potência, conforme proposto em [1]. O objetivo central é encontrar um cronograma de manutenção ótimo que minimize os riscos operacionais (como sobrecargas e violações de tensão), garantindo que o sistema permaneça estável mesmo sob cenários de contingência (critério N-1)

3.4) Arquitetura e ferramentas

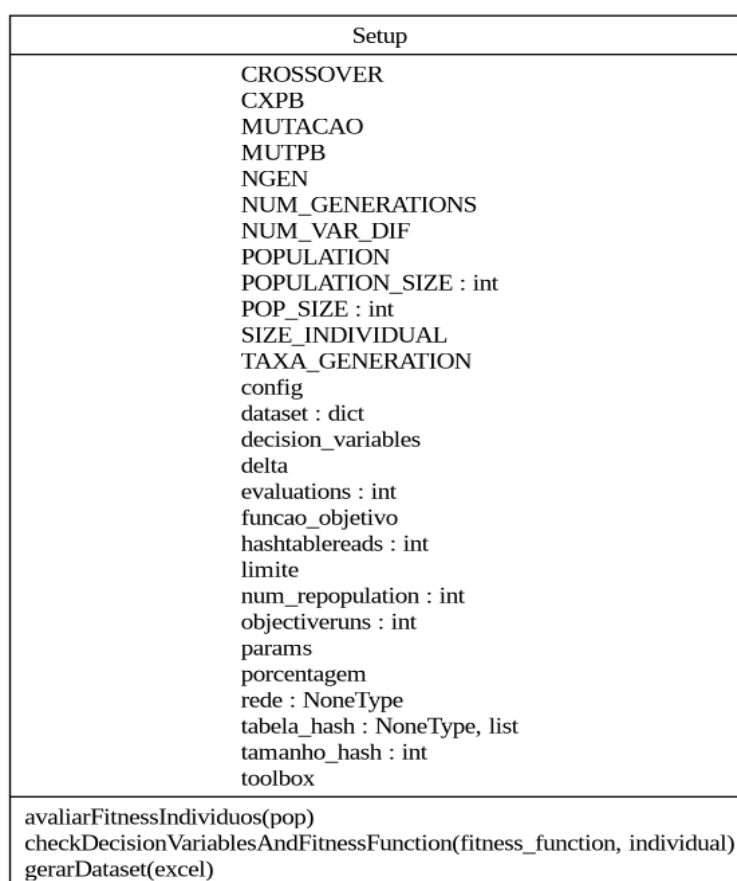
O projeto integra as seguintes tecnologias:

- **Otimização:** Algoritmo Genético (AG) implementado com a biblioteca *DEAP*.
- **Simulação de Rede:** Análise de fluxo de potência realizada com *Pandapower*, que serve como a função de *fitness* (avaliação) para o AG.

- **Interface de Configuração:** Um *launcher* de *desktop* desenvolvido com *QT 6* ou *PySide6* para facilitar a configuração de múltiplos cenários e execuções.
- **Dashboard Web para Análise:** Uma aplicação web interativa com *Streamlit* para visualização e análise detalhada dos resultados.

A classe *Setup* assim como desenvolvida em [6] funciona em conjunto com a classe *AlgEvolutivoRCE* que executa o AG. No projeto anterior, a otimização funcionava via interface de linha de comando, mas no projeto atual foi desenvolvida uma nova camada chamada *dashboard* para visualização de dados dos resultados com uma interface fácil e intuitiva para o usuário final. No diagrama da Figura 1 é apresentado o UML da classe *Setup* desenvolvida para parametrizar e configurar a otimização.

Figura 1: Class *Setup*



Fonte: Autoria Própria usando *pyreverse*

Na figura 2 é mostrado o diagrama da classe principal usando algoritmos genéticos com a estratégia RCE [7]. Após estudos bibliográficos e leitura da documentação, foi averiguado que o AG é considerado um algoritmo de busca estocástica, onde os estados sucessores são criados a partir da combinação de estados

anteriores para executar a sua busca por uma solução ótima. Essa característica demanda realizar diversas execuções da otimização com diferentes sementes para o gerador de números pseudo-aleatórios. No trecho de Código 1 é apresentado um exemplo ilustrativo de como foi possível implementar diferentes configurações com um número ajustável de execuções.

Figura 2: Class *AlgoritmoEvolutivoRCE*

AlgoritmoEvolutivoRCE
CONJUNTO_ELITE_RCE : set DEBUG : bool POPULATION allFitnessValues : dict allIndividualValuesArray : list best_individual_array : list best_solutions_array : list dashboard data : dict decision_variables : list fitness_function hof logbook pop_RCE : list repopulation_counter : int setup stats validateCounter : int
aplicar_RCE(generation, current_population) calculateFitnessGeneration(new_pop) checkClonesInPop(ind, new_pop) cout(msg) criterios_RCE(population) elitismoSimples(pop) generateInfoIndividual(new_pop, generation) registrarDados(generation) run(RCE, num_pop) show_ind_df(array, text, save) visualizarPopAtual(geracaoAtual, stats)

Fonte: Autoria Própria usando *pyreverse*

Código 1: Exemplo com programação procedural inicial que realiza a otimização fazendo múltiplas execuções

Python

...

```
if __name__ == "__main__":

    configuracoes_execucoes = {
        "key": True,
        "value": 5,
        "parametros_opcionais": [
            {"MUTACAO": [90, 80, 70]},
            {"CROSSOVER": [90, 80, 70]},
            {'NUM_GENERATIONS': [100, 200, 300]},
        ]
    }

    # Inicializa a configuração do algoritmo
    configuracao_algoritmo = Setup(params, rastrigin)

    # Inicializa o algoritmo
    algoritmo_evolutivo = AlgoritmoEvolutivoRCE(configuracao_algoritmo, DEBUG=False)

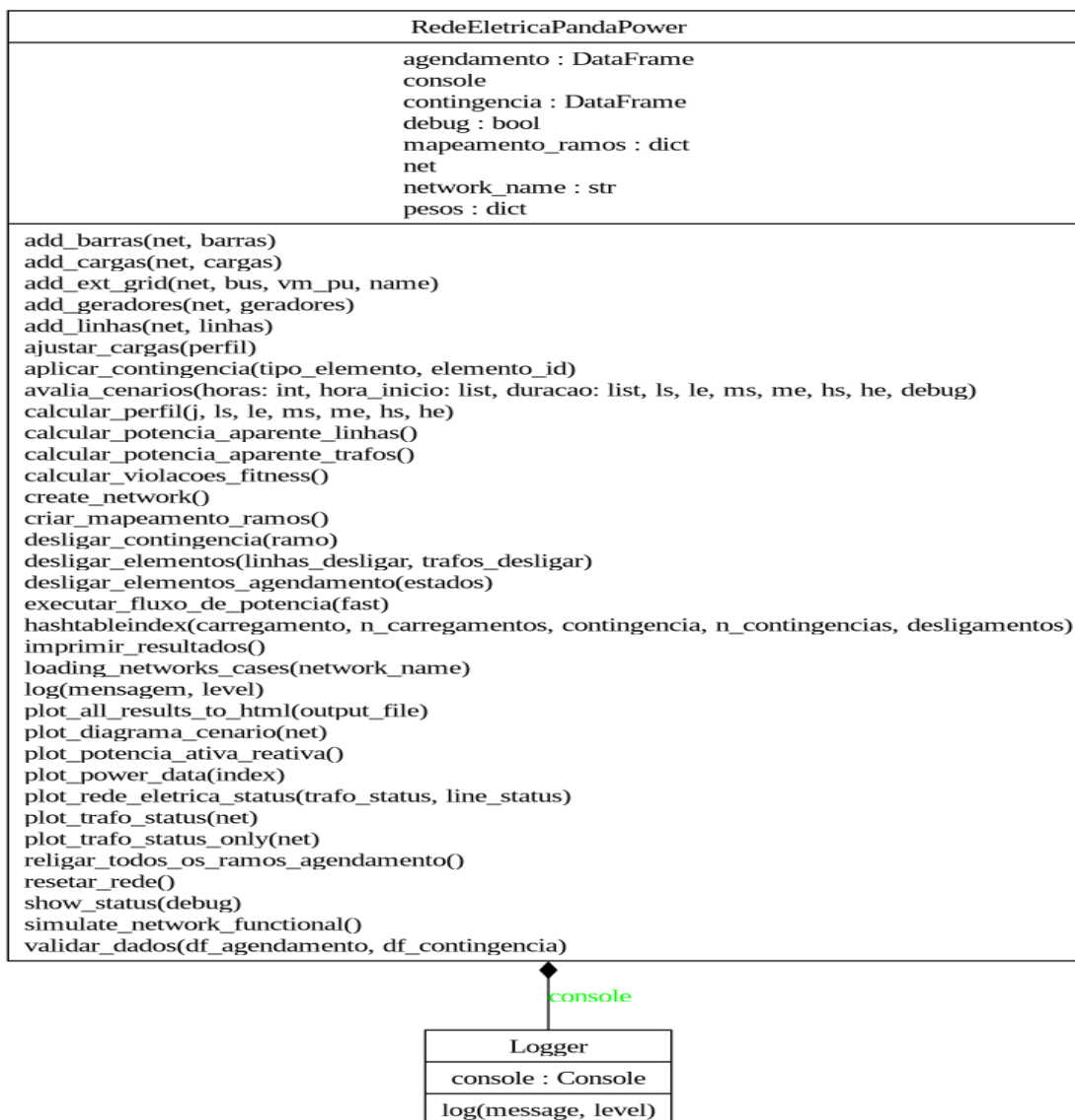
    # Inicializa o dashboard
    painel_controle = algoritmo_evolutivo.dashboard.visualize(configuracao_algoritmo)

    # Carrega dados de execuções
    load_many_executions(configuracoes_execucoes, painel_controle, algoritmo_evolutivo)
```

Fonte: Autoria própria

A função *load_many_executions* utiliza uma biblioteca nativa do python chamada *itertools* para criar diferentes tipos de combinações no script *run.py* como *run.py -config_num 1 exec_num 10* usando o parâmetro inicial em json e faz um loop para cada configuração e avaliando cada cenário da rede selecionada. A Figura 3 representa o diagrama da modelagem para os casos de uso do SEP usando *pandapower*.

Figura 3: Classe Rede Elétrica usando *PandaPower*



Fonte: Autoria Própria usando *pyreverse*

O *framework* foi construído sobre uma arquitetura modular MVC (*Model-View-Controller*) [8], utilizando princípios de *clean code* e *Design Patterns*, que separa as responsabilidades em três componentes principais:

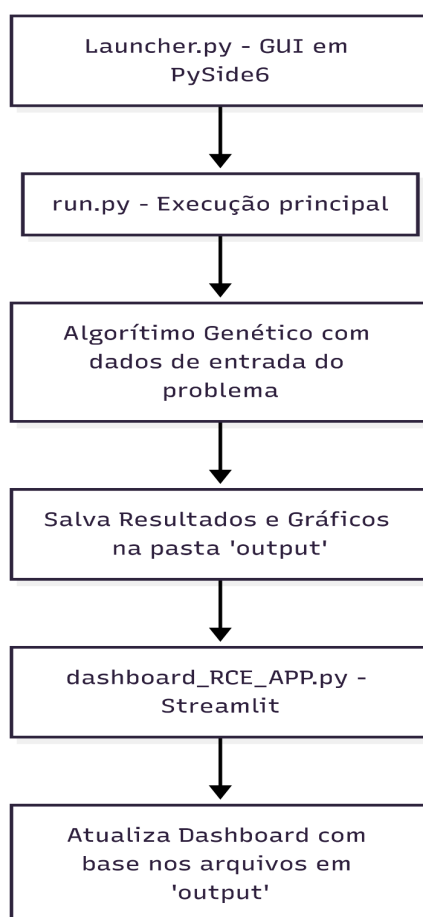
1. **Launcher (Frontend de Configuração):** Interface gráfica onde o usuário define os parâmetros do algoritmo genético usando o arquivo *params.json* e as variações para a bateria de testes *options.json*. Possui também um aba de logs e botões conectados ao *dashboard*.

2. **Backend:** Script principal *run.py* que orquestra todo o processo de otimização. Ele lê as configurações, executa o algoritmo evolutivo e salva os resultados usando ferramentas nativas do *Python*, *itertools*, sub-rotinas e *subprocess*.

3. **Dashboard (Frontend de Visualização):** Aplicação *web* em *Streamlit* que lê os dados gerados pela execução, consolida-os e apresenta gráficos, tabelas e estatísticas para análise. Os dados são consumidos pelos resultados na pasta *src/output/* onde cada resultado de execução é armazenado em *cache*.

Na Figura 4, é representado o passo a passo da simulação separado pelos arquivos *Python* separados e rodando em *subprocess*.

Figura 4 - Fluxograma da simulação

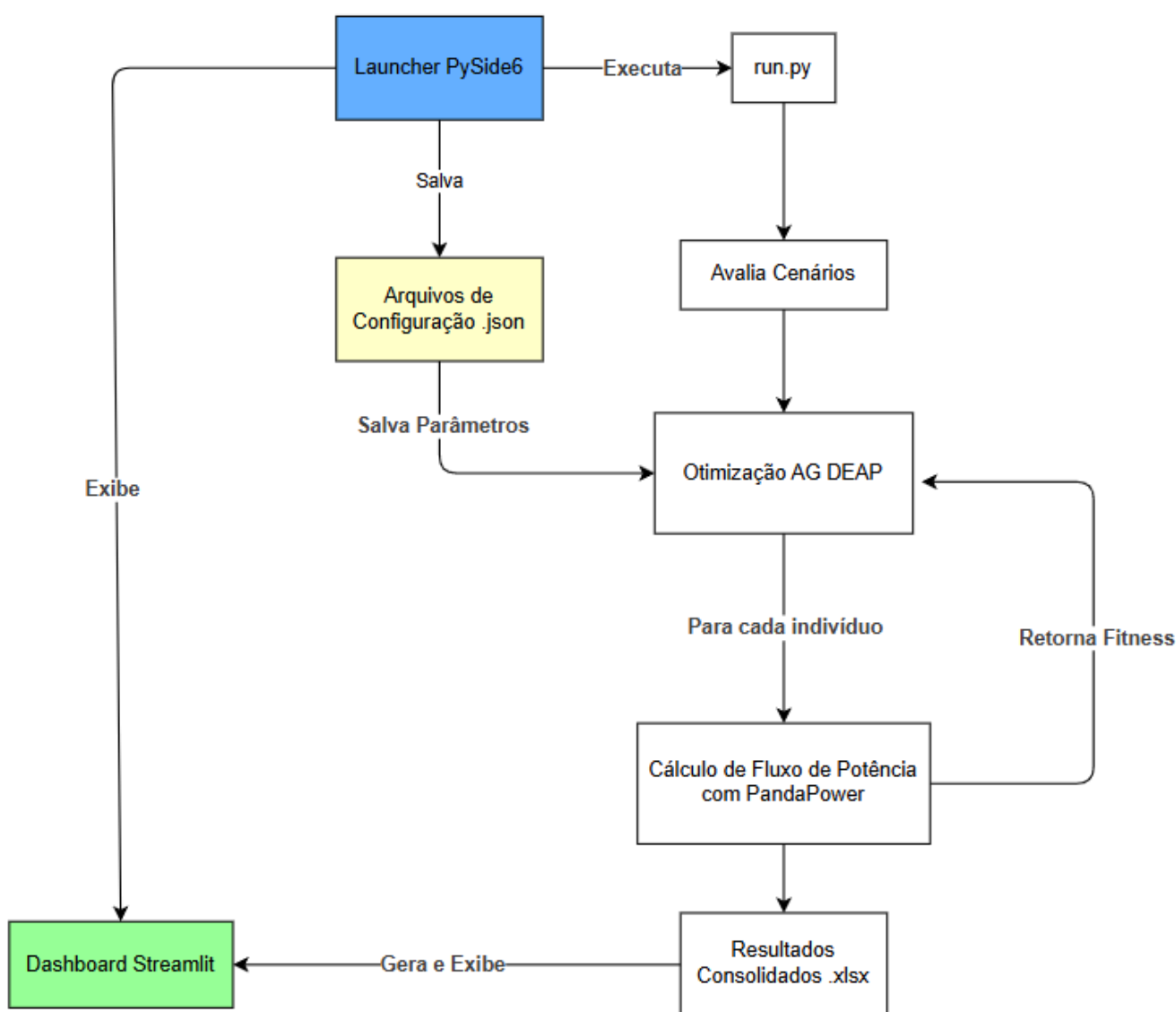


Fonte: Autoria própria

A Figura 5 ilustra as relações das classes implementadas voltadas para a experiência do usuário passando pelo *launcher*. O quadrado de cor azul representa o *launcher.py* que serve como GUI do programa.

Esta parte se conecta tanto com os arquivos de configuração *params.json* e *options.json*, em amarelo, quanto no *backend* da aplicação. Essa conexão com o *backend* é realizada pelo *script run.py* onde utiliza as classes *Setup* e *AlgEvolutivoRCE*. O algoritmo evolutivo usa a função objetivo que executa o *PandaPower* para avaliar todos os cenários. Todos os diagramas de classes foram gerados utilizando a ferramenta *pyreverse*, uma biblioteca para o *Python* e *shell script* para automação de processos.

Figura 5: O relacionamento entre as Classes criadas



Fonte: Autoria própria

Ao abrir o *Launcher Desktop*, o usuário tem acesso às abas onde pode configurar o AG, definindo múltiplos valores para parâmetros como `'MUTACAO'`, `'CROSSOVER'`, `'POP_SIZE'` e

`NUM\_GENERATIONS` para as diferentes execuções. O *framework* irá gerar e executar todas as combinações de configurações possíveis. Ao clicar em "Executar", o *Launcher* invoca o *script run.py* em um processo separado no terminal, aumentando a confiabilidade da interface contra travamentos. O *script* de execução inicia o processo do Algoritmo Genético onde uma população inicial de "indivíduos" é gerada. Cada indivíduo representa uma solução candidata para o problema (ex: um vetor com os horários de início para cada desligamento). Para cada indivíduo da população, o sistema executa uma análise dos cenários para o agendamento traduzido em uma série de cenários de rede a serem testados, considerando os desligamentos programados, perfis de carga (leve, média, pesada) e contingências (falhas N-1). Antes de rodar uma simulação completa, o *framework* verifica se o cenário (uma combinação única de topologia e carga) já foi avaliado anteriormente. Se sim, o resultado (*fitness*) é lido diretamente de uma tabela *hash*, evitando o recálculo do fluxo de potência. Se o cenário é inédito, o *Pandapower* é utilizado para executar o cálculo de fluxo de potência. O resultado do fluxo de potência é analisado para verificar violações de limites operacionais (tensão nas barras, carregamento de linhas e transformadores). As violações são quantificadas, ponderadas por pesos pré-definidos e somadas para gerar um único valor de *fitness* que representa a "qualidade" da solução. O objetivo do AG é encontrar a solução que minimiza a função objetivo definida em [2]. Para este fim, o *DEAP* aplica os operadores genéticos e o *framework* utiliza a técnica de RCE [1], uma forma avançada de elitismo onde um conjunto das melhores soluções já encontradas é mantido e usado para repovoar uma parte da população periodicamente, evitando a convergência prematura para ótimos locais e permitindo diversidade em soluções. O ciclo de evolução se repete pelo número de gerações definido pelo usuário. Ao final, a melhor solução encontrada é registrada. Os resultados detalhados de cada execução (*logbook* da evolução, parâmetros, melhor indivíduo) são salvos em arquivos *.json* no diretório `src/output/`. Um script de consolidação *consolidar_resultados.py* é executado para ler todos os arquivos de resultado individuais e criar uma única planilha excel em formato *.xlsx* com o resumo de toda a bateria de testes. O *Dashboard* em *Streamlit* lê tanto os arquivos detalhados quanto o consolidado para apresentar os dados de forma interativa, com gráficos de convergência, tabelas comparativas e detalhes de cada execução. O *Framework* cobre todo o ciclo de vida da experimentação, da configuração à análise de resultados, abstraindo a complexidade do usuário. O uso de uma tabela *hash* para armazenar resultados de cenários já calculados torna a análise de um vasto número de contingências computacionalmente viável. Esta redução de tempo computacional se torna mais evidente em simulações de sistemas mais complexos

3.5) Exemplo de implementação de uma função objetivo para um SEP

O pacote *PandaPower* possui atributos da rede como **cargas, reatores, barramentos, linhas e transformadores**. É possível simular uma rede colocando seus dados de entrada conforme seus atributos. No trecho de Código 2 é um exemplo básico de uso do *PandaPower* criando uma nova rede de 12 barras onde é possível colocar os dados de entrada e ter como resultado as tensões nas barras, correntes das linhas/trafos e potência ativa e reativa das cargas. No trecho de Código 3 é realizada a criação de cada elemento da rede,

executando seu fluxo de potência e consolidando seus resultados em relatório em *.html* para facilitar futuras implementações utilizando esta ferramenta.

Código 2: Criação de uma SEP de 12 barras

Python

```
# Dados para as barras (12 barras: 6 à esquerda, 6 à direita)
barras = [
    {"id": 0, "nome": "Barra 1E", "tensao": 1.05}, # Barra de referência (Slack)
    {"id": 1, "nome": "Barra 2E", "tensao": 1.02},
    {"id": 2, "nome": "Barra 3E", "tensao": 20.0},
    {"id": 3, "nome": "Barra 4E", "tensao": 20.0},
    {"id": 4, "nome": "Barra 5E", "tensao": 20.0},
    {"id": 5, "nome": "Barra 6E", "tensao": 20.0},
    {"id": 6, "nome": "Barra 1D", "tensao": 20.0},
    {"id": 7, "nome": "Barra 2D", "tensao": 20.0},
    {"id": 8, "nome": "Barra 3D", "tensao": 20.0},
    {"id": 9, "nome": "Barra 4D", "tensao": 20.0},
    {"id": 10, "nome": "Barra 5D", "tensao": 20.0},
    {"id": 11, "nome": "Barra 6D", "tensao": 20.0},
]

# Linhas conectando as barras em dois ramos (esquerdo e direito) e uma ligação central
linhas = [
    # Lado Esquerdo (E)
    {"de": 0, "para": 1, "r_ohm_per_km": 0.01, "x_ohm_per_km": 0.03, "c_nf_per_km": 10, "max_i_ka": 0.2, "comprimento_km": 1.0},
    {"de": 1, "para": 2, "r_ohm_per_km": 0.01, "x_ohm_per_km": 0.03, "c_nf_per_km": 10, "max_i_ka": 0.2, "comprimento_km": 1.0},
    {"de": 2, "para": 3, "r_ohm_per_km": 0.01, "x_ohm_per_km": 0.03, "c_nf_per_km": 10, "max_i_ka": 0.2, "comprimento_km": 1.0},
    {"de": 3, "para": 4, "r_ohm_per_km": 0.01, "x_ohm_per_km": 0.03, "c_nf_per_km": 10, "max_i_ka": 0.2, "comprimento_km": 1.0},
    {"de": 4, "para": 5, "r_ohm_per_km": 0.01, "x_ohm_per_km": 0.03, "c_nf_per_km": 10, "max_i_ka": 0.2, "comprimento_km": 1.0},

    # Lado Direito (D)
    {"de": 6, "para": 7, "r_ohm_per_km": 0.01, "x_ohm_per_km": 0.03, "c_nf_per_km": 10, "max_i_ka": 0.2, "comprimento_km": 1.0},
    {"de": 7, "para": 8, "r_ohm_per_km": 0.01, "x_ohm_per_km": 0.03, "c_nf_per_km": 10, "max_i_ka": 0.2, "comprimento_km": 1.0},
    {"de": 8, "para": 9, "r_ohm_per_km": 0.01, "x_ohm_per_km": 0.03, "c_nf_per_km": 10, "max_i_ka": 0.2, "comprimento_km": 1.0},
    {"de": 9, "para": 10, "r_ohm_per_km": 0.01, "x_ohm_per_km": 0.03, "c_nf_per_km": 10, "max_i_ka": 0.2, "comprimento_km": 1.0},
    {"de": 10, "para": 11, "r_ohm_per_km": 0.01, "x_ohm_per_km": 0.03, "c_nf_per_km": 10, "max_i_ka": 0.2, "comprimento_km": 1.0},

    # Ligação central entre os dois lados
    {"de": 2, "para": 8, "r_ohm_per_km": 0.02, "x_ohm_per_km": 0.04, "c_nf_per_km": 15, "max_i_ka": 0.2, "comprimento_km": 1.5},
    {"de": 5, "para": 11, "r_ohm_per_km": 0.02, "x_ohm_per_km": 0.04, "c_nf_per_km": 15, "max_i_ka": 0.2, "comprimento_km": 1.5},
]
```

Fonte: Autoria própria

Código 3: SimulateNewNetwork.py

Python

```
import pandapower as pp
import pandas as pd
import numpy as np
import plotly.graph_objects as go
from IPython.display import display

def simulate_NEW_network():
    # Criando a rede elétrica vazia
    net = pp.create_empty_network()

    # Criando as barras
    for barra in barras:
        pp.create_bus(net, name=barra["nome"], vn_kv=barra["tensao"], index=barra["id"])

    # Criando a barra de referência (Slack)
    pp.create_ext_grid(net, bus=0, vm_pu=1.0, name="Slack")

    # Criando as linhas
    for linha in linhas:
        pp.create_line_from_parameters(
            net,
            from_bus=linha["de"],
            to_bus=linha["para"],
            length_km=linha["comprimento_km"],
            r_ohm_per_km=linha["r_ohm_per_km"],
            x_ohm_per_km=linha["x_ohm_per_km"],
            c_nf_per_km=linha["c_nf_per_km"],
            max_i_ka=linha["max_i_ka"],
            name=f"Linha {linha['de']} -> {linha['para']}",
        )

    # Dados fictícios para as cargas
    cargas = [
        {"bus": 1, "p_mw": 0.02, "q_mvar": 0.01, "nome": "Carga 1"},
        {"bus": 2, "p_mw": 0.03, "q_mvar": 0.015, "nome": "Carga 2"},
    ]

    # Criando as cargas
    for carga in cargas:
        pp.create_load(net, bus=carga["bus"], p_mw=carga["p_mw"], q_mvar=carga["q_mvar"], name=carga["nome"])

    # Dados fictícios para os geradores
    geradores = [
        {"bus": 3, "p_mw": 0.05, "vm_pu": 1.02, "nome": "Gerador PV"},
    ]

    # Criando os geradores
    for gerador in geradores:
        pp.create_sgen(net, bus=gerador["bus"], p_mw=gerador["p_mw"], vm_pu=gerador["vm_pu"],
            name=gerador["nome"])

    # Executando o fluxo de potência
    pp.runpp(net)

    # Exibindo os resultados
    print(f"Resultados de Sistema Elétrico de Potência com {len(barras)} Barras:")
    print("""
    VM_PU = Tensões em pu em cada barra
    va_degree = Angulo de fase em graus
    p_mw = Potencia Aparente em MW
    q_mvar = Potencia Reativa em MW

    + -> Potencia Demandada
    - -> Potencia Gerada
    """)
    display(net.res_bus)
```



```

print("\nResultados das Linhas:")
display(net.res_line)

print("\nResultados das Cargas:")
display(net.res_load)

return net

def plot_results(net):
    # Plot das tensões nas barras
    fig = go.Figure()
    fig.add_trace(go.Bar(x=net.res_bus.index, y=net.res_bus.vm_pu, name="Tensão (pu)"))
    fig.update_layout(title="Tensões nas Barras", xaxis_title="Barras", yaxis_title="Tensão (pu)")
    fig.show()

    # Plot das correntes nas linhas
    fig = go.Figure()
    fig.add_trace(go.Bar(x=net.res_line.index, y=net.res_line.i_ka, name="Corrente (kA)"))
    fig.update_layout(title="Correntes nas Linhas", xaxis_title="Linhas", yaxis_title="Corrente (kA)")
    fig.show()

    # Plot das potências nas cargas
    fig = go.Figure()
    fig.add_trace(go.Bar(x=net.res_load.index, y=net.res_load.p_mw, name="Potência Ativa (MW)"))
    fig.update_layout(title="Potências Ativas nas Cargas", xaxis_title="Cargas", yaxis_title="Potência Ativa (MW)")
    fig.show()

    fig.to_html("./network_results.html")

def main_simulate():
    net = simulate_NEW_network()
    plot_results(net)

main_simulate()

```

Fonte: Autoria própria

3.6) Caso de uso para o problema de agendamento ótimo na rede IEEE 14

Para realizar a otimização de SEP, é feito um algoritmo para avaliar os cenários e calcular o fluxo de potência. Foi escolhido como caso de uso inicial a rede IEEE 14 por ser bem difundida, presente no *PandaPower* e representar o caso mais rápido para as simulações. Foi utilizada a tabela de contingências e tabela de horários de agendamentos sugerida em [1], onde os horários iniciais são as variáveis de decisão para o problema. Os valores possíveis para as variáveis foram limitados entre 0 e 31. Para representar os horários do dia posterior (maiores que 23) é utilizado um *.

Na classe desenvolvida e representada na Figura 6, a função objetivo possui instruções para avaliar as violações presentes em todos os cenários considerando a lista de contingências, através do cálculo do fluxo de potência. O trecho de Código 4 ilustra os métodos criados da classe *RedeEletricaPandaPower* numa função usada como subrotina para todos os outros casos de redes IEEE.

Código 4: Sub Rotina para análise de contingências para *RedeEletricaPandaPower*

Python

```
def analise_contingencias_SEP(rede, setupobj, matriz_cenarios , agendamento_df, contingencia_df):
    """
    Executa a análise de contingências para um determinado agendamento de manutenção.

    """
    contingencias = contingencia_df['contingencia'].to_list()
    num_carregamentos = 3
    num_contingencias = len(contingencias)
    num_desligamentos = len(agendamento_df)
    violacoes_total = []

    contingencias_selecionadas = {
        "ramos": [],
        "contingencia": []
    }

    try:
        for cenario in matriz_cenarios:
            perfil = cenario[0]
            estado_ramos = cenario[1:]
            rede.ajustar_cargas(perfil)

            for contingencia_atual in range(1, num_contingencias + 1):
                hash_key = rede.hashtableindex(perfil, num_carregamentos, contingencia_atual,
                num_contingencias, estado_ramos)

                if setupobj.tabela_hash[hash_key] < 0.0:
                    rede.religar_todos_os_ramos_agendamento()
                    rede.desligar_elementos_agendamento(estado_ramos)

                    ramo_contingencia = list(contingencia_df.loc[contingencia_df['contingencia'] ==
                    contingencia_atual, ['from', 'to']].values[0])
                    rede.desligar_contingencia(ramo_contingencia)

                    if ramo_contingencia not in contingencias_selecionadas["ramos"]:
                        contingencias_selecionadas["ramos"].append(ramo_contingencia)
                        contingencias_selecionadas["contingencia"].append(contingencia_atual)

                    if rede.executar_fluxo_de_potencia():
                        fitness, _ = rede.calcular_violacoes_fitness()
                    else:
                        fitness = rede.pesos.get("demanda", 9999)
                        setupobj.tabela_hash[hash_key] = fitness
                        setupobj.objectiveruns += 1
                    else:
                        fitness = setupobj.tabela_hash[hash_key]
                        setupobj.hashtablereads += 1

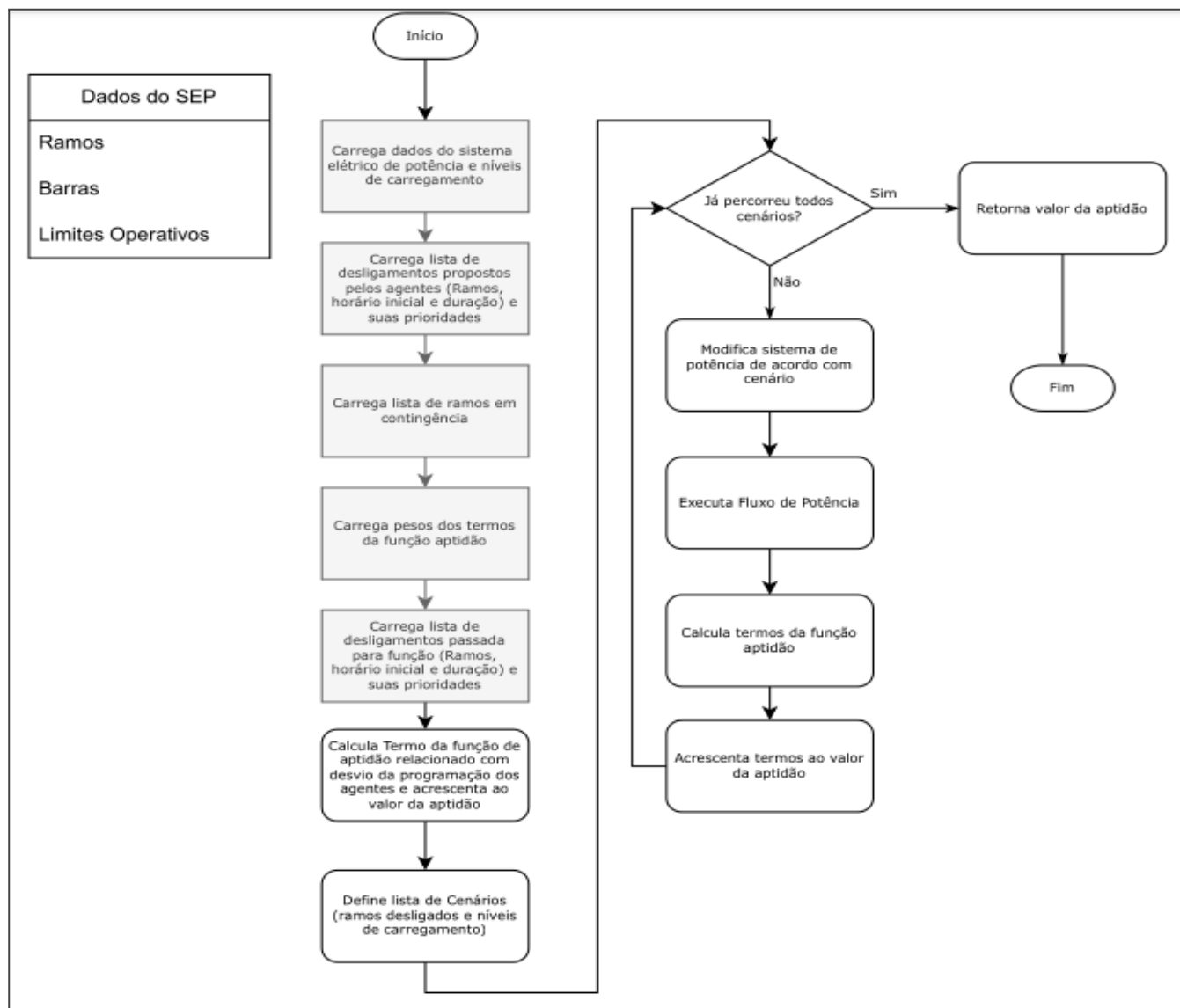
                    violacoes_total.append(fitness)

                fitness_final = sum(violacoes_total)
                return fitness_final, contingencias_selecionadas

    except Exception as e:
        print(f"\nErro durante a análise de contingências: {e}")
        return float('inf'), {}
```

Fonte: Autoria própria

Figura 6: Fluxograma função objetivo para SEP



Fonte: Autoria Própria

4) PRODUÇÃO TÉCNICO-CIENTÍFICA

A produção técnico-científica deste trabalho consiste no desenvolvimento e implementação dos seguintes algoritmos disponíveis publicamente:

- Algoritmo de Cálculo de Violações:** Implementado para identificar e avaliar o impacto de cada cenário de agendamento na operação da rede, quantificando as violações de limites operacionais (ex: sobrecarga, violação de tensão), de acordo com [1].

- **Análise para Cada Cenário Possível:** Metodologia implementada para analisar o desempenho do agendamento em diferentes cenários operacionais, incluindo variações nos horários de agendamento e ocorrência de contingências.
- **Análise de contingências de casos base IEEE usando *PySide6***
- **Dashboard em *Streamlit* para análise de resultados**
- **Framework com múltiplas execuções usando *DEAP* e *PandaPower* usando *AG* para *SEP***

Está em produção um artigo para congresso científico nacional que demonstra o uso do *framework* em um caso de uso do problema do agendamento ótimo de intervenções em redes elétricas.

5) RESULTADOS

O *framework* desenvolvido neste projeto foi testado utilizando os sistemas teste IEEE e um sistema de teste adicional (SIN 45), com diferentes configurações do Algoritmo Genético (AG) e da técnica RCE (Repopulação Conjunto Elite). As simulações foram executadas em um ambiente com *CPU* Ryzen 2600 (3,40 GHz) com 6 núcleos e 12 *threads*, *GPU* RX 570 4GB VRAM, 24 GB RAM e SSD. Foram realizadas variações de parâmetros como taxa de **mutação**, **crossover**, **tamanho de população** e **número de gerações** em 4 configurações diferentes. A implementação permitiu calcular a função objetivo para cada agendamento proposto, avaliando violações operacionais (sobrecarga de linhas/transformadores e violações de tensão) em múltiplos cenários (passagem de carga leve, média e pesada e contingências N-1). A utilização de uma tabela *hash* para armazenar resultados de cenários já avaliados reduziu significativamente o número de execuções custosas do fluxo de potência, mantendo a exatidão dos cálculos.

O programa em *Python* desenvolvido inclui a função objetivo definida para as instâncias de teste definidas em [1], utilizando os sistemas IEEE14, IEEE 30, IEEE 57 e IEEE 118. A classe utilizada pela função objetivo implementada (ilustrada na Figura 1) demonstrou-se funcional e capaz de calcular as violações de um agendamento para diversos cenários. O algoritmo desenvolvido permite analisar o impacto de diferentes programações de manutenção na operação da rede, identificando as violações de limites operacionais, como não atendimento à demanda, sobrecarga de linhas e transformadores e violações de tensão nas barras. Os cenários de interesse para a análise incluem diferentes programações de desligamento para manutenção, bem como a consideração de contingências (falhas inesperadas em componentes da rede) que podem ocorrer durante o período de manutenção. A combinação de diferentes programações de desligamento com diferentes cenários de contingência permite avaliar a o agendamento proposto, conforme estipulado em [1]. Durante os testes foram utilizadas as redes elétricas IEEE14, 30, 57 e 118 pré-configuradas na biblioteca *PandaPower* e foi criada a rede SIN45 conforme [1]. Para a carga média, foi adotado o ponto de operação definido ou criado na biblioteca para estes sistemas. Para a carga leve foi adotado um ajuste de 0,941 nas cargas e geração. Para a carga pesada, o valor de ajuste de 1,177 foi adotado.

Durante os testes, foi verificado que o sistema IEEE 57 implementado na biblioteca apresentava muitas diferenças com o sistema implementado em [2]. Foi decidido eliminar este sistema das instâncias de teste de análise.

O AG gera diversas soluções (individuais) a cada geração. Cada solução é um cronograma de desligamentos. É comum que cronogramas diferentes acabem gerando cenários idênticos em determinados momentos. Por exemplo, dois cronogramas podem ter as mesmas 3 linhas desligadas às 15h em uma carga pesada. Para impedir que a função aptidão do AG seja (re-)executada nestes casos, foi implementada a estratégia proposta em [1], armazenando os resultados da aptidão por cenário em uma tabela hash. Esta tabela permanece em memória durante as execuções e é salva em uma planilha Excel para garantir sua permanência entre execuções do AG para uma mesma função objetivo. Considerando que a consulta em memória do valor da função aptidão do cenário tem custo computacional muito menor que a execução da função aptidão, esta implementação reduziu o custo computacional de execução do framework. Também é possível armazenar o arquivo Excel e reaproveitá-lo em futuros usos da ferramenta. No trecho de Código 5 é demonstrado como funciona o *loop* de combinações e o uso da sub rotina para consulta na tabela *hash* para cada configuração.

Código 5: Implementação do script de simulação usando hash table

Python

```
# Imports principais do framework
from AlgEvolutivoRCE_backup.Setup import Setup
from AlgEvolutivoRCE_backup.alg_evolutivo_rce import AlgoritmoEvolutivoRCE
#! Importando a minha função objetivo dentro do projeto
from utils.functions_fitness.function_IEEE_14_contingencias import funcao_objetivo_IEEE14, get_hash_table_size

print(f"\n\nIniciando configuração {config_num} com os params.json:\n{params}\n")

setup = Setup(
    params,
    fitness_function=funcao_objetivo_IEEE14,
    tamanho_hash=get_hash_table_size()
)
print("Classe Setup iniciada para a configuração.")

consultaHashTable()

for exec_num in range(1, repeticoes + 1):
    print(f"\n--- Iniciando execução {exec_num}/{repeticoes} ---")

    alg = AlgoritmoEvolutivoRCE(setup, DEBUG=False)
    print("Algoritmo Evolutivo iniciado.")

    pop_with_repopulation, logbook_with_repopulation, best_individual, _ = alg.run(RCE=True)
    print("\nEvolução concluída - 100%")
```

```
best_solution_generation, best_solution_variables, best_solution_fitness, grafico_RCE =
alg.dashboard.visualize(
    logbook_with_repopulation,
    pop_with_repopulation,
    config_num=config_num,
    execution_num=exec_num,
)
```

Fonte: Autoria Própria

Na figura 7 é ilustrado o elemento terminal da tela do Launcher durante a execução de 4 configurações com os parâmetros AG configurados no *frontend*. Neste terminal pode ser visto o tempo necessário para cada execução e o tempo total de todas as execuções, com o registro do número de vezes em que o algoritmo calculou a função aptidão ou simplesmente acessou a tabela hash ao visitar um cenário ainda não calculado.

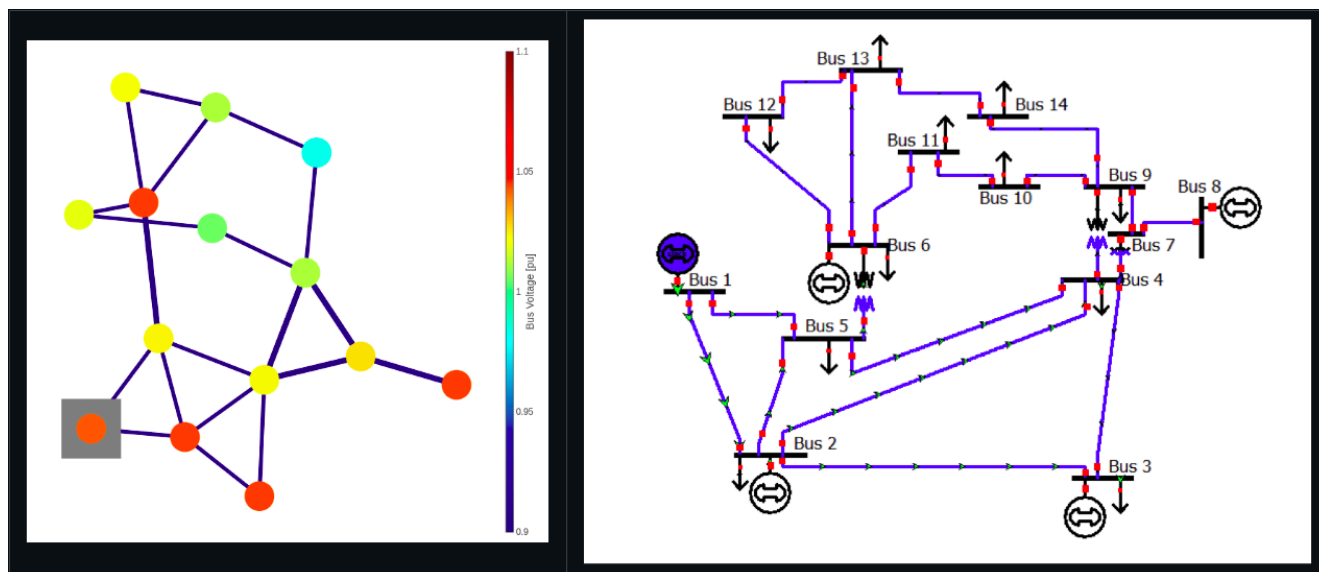
Figura 7: Exemplo de terminal do *frontend* durante a execução mostrando custo computacional

```
[14:17:24] --- Iniciando execução 7/10 ---
[14:17:24] Algoritmo Evolutivo iniciado. DEBUG MODE = False
[14:17:24]
[14:17:24] Evolução concluída - 100%
[14:17:24]
[14:17:24] =====
[14:17:24] >>> Soluções do problema (Execução 7 - Console Output) <<<
[14:17:24] =====
[14:17:24] Best Generation: 17
[14:17:24] Best Variables: [23, 28, 28, 29, 29, 16, 29, 9, 0, 0]
[14:17:24] Best Fitness: 13.567487542229063
[14:17:24] =====
[14:17:24]
[14:17:24] Duração desta Execução: 3 minutos 17 segundos
[14:17:24] Tempo Total Acumulado: 1 horas 28 minutos 30 segundos
[14:17:24] Objective functions runs: 2229
[14:17:24] Consultas HashTable: 38958
[14:17:24]
[14:17:24] Resultados e visualizações salvos com sucesso.
[14:17:24]
```

Fonte : Autoria própria

A criação da função objetivo e os testes foram realizados inicialmente na rede IEEE 14 por possuir apenas 5 horários de agendamentos [1] e menor número de barras e ramos. Na Figura 8, é ilustrado um diagrama da rede com sua topologia e um exemplo de diagrama realizado pelo objeto *plotting* do *PandaPower* identificando as tensões em cada barramento.

Figura 8: Exemplo de simulação da Rede IEEE 14 com plot do *PandaPower*



Fonte: Autoria própria e diagrama disponível em [1]

Para ilustrar a jornada do usuário pelo *frontend*, são ilustradas a seguir exemplos de telas vistas e utilizadas para todas as simulações apresentadas nas seções seguintes. Os exemplos aqui apresentados foram obtidos em ambiente *Windows*, mas a implementação admite a execução em outros sistemas operacionais, tendo sido testada também em ambiente *Linux* e com interface de linha de comando.

Inicialmente, o usuário resgata o repositório do *github* e executa localmente o arquivo “*instalar.bat*” no diretório raiz do projeto. Este arquivo é responsável por instalar todos os requisitos para execução do projeto e apresenta visual semelhante àquele que pode ser visto ilustrado na Figura 9. O usuário avança e verifica o registro de todas as dependências instaladas. Em sequência, executa o botão “Iniciar” para executar o *Launcher* e conclui o instalador, fechando sua janela.

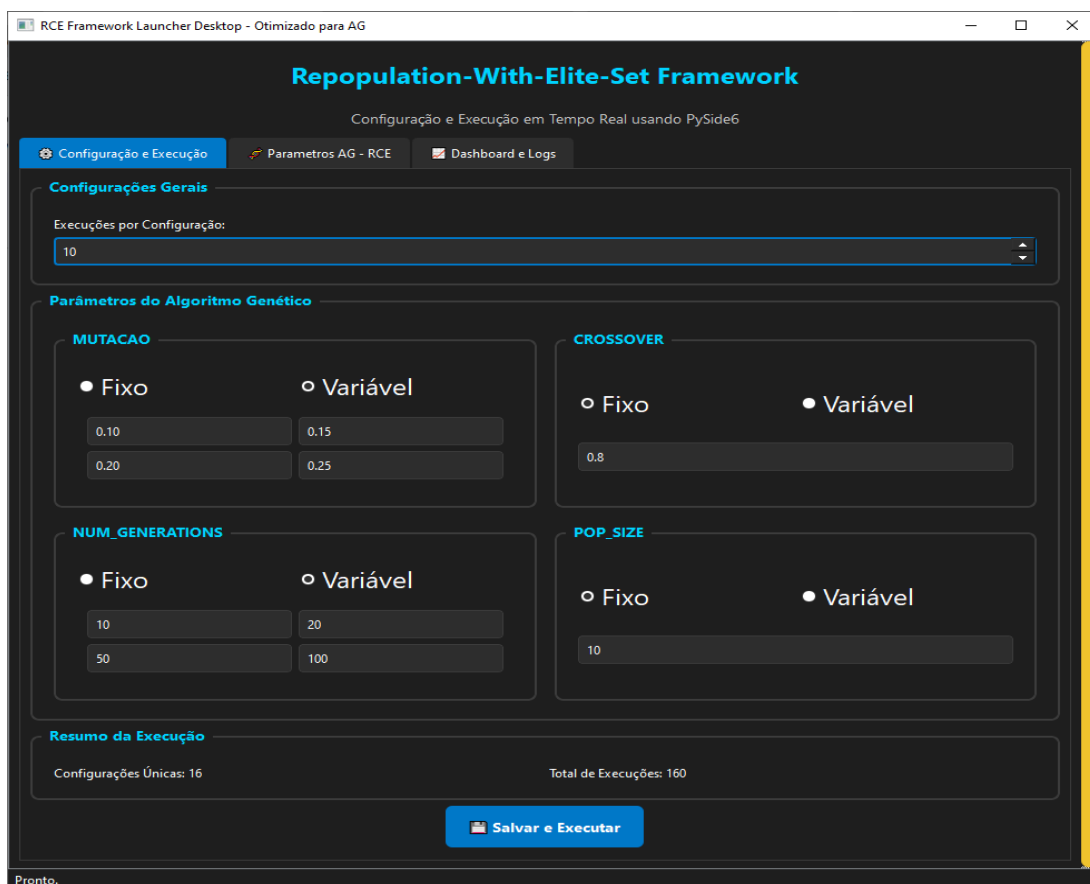
Figura 9: Tela inicial do script instalador



Fonte: Autoria própria

Na tela inicial do Launcher, é possível realizar a parametrização de diversas configurações na aba “Configuração e Execução”, assim como ilustrado na Figura 10. Nas opções selecionadas neste exemplo ilustrado, são escolhidos 4 parâmetros diferentes para mutação e 4 parâmetros para o número de gerações, perfazendo um total de $4 \times 4 = 16$ configurações únicas. Considerando que foram selecionadas 10 execuções por configuração, é possível verificar um total de 160 execuções para o exemplo ilustrado na Figura 10.

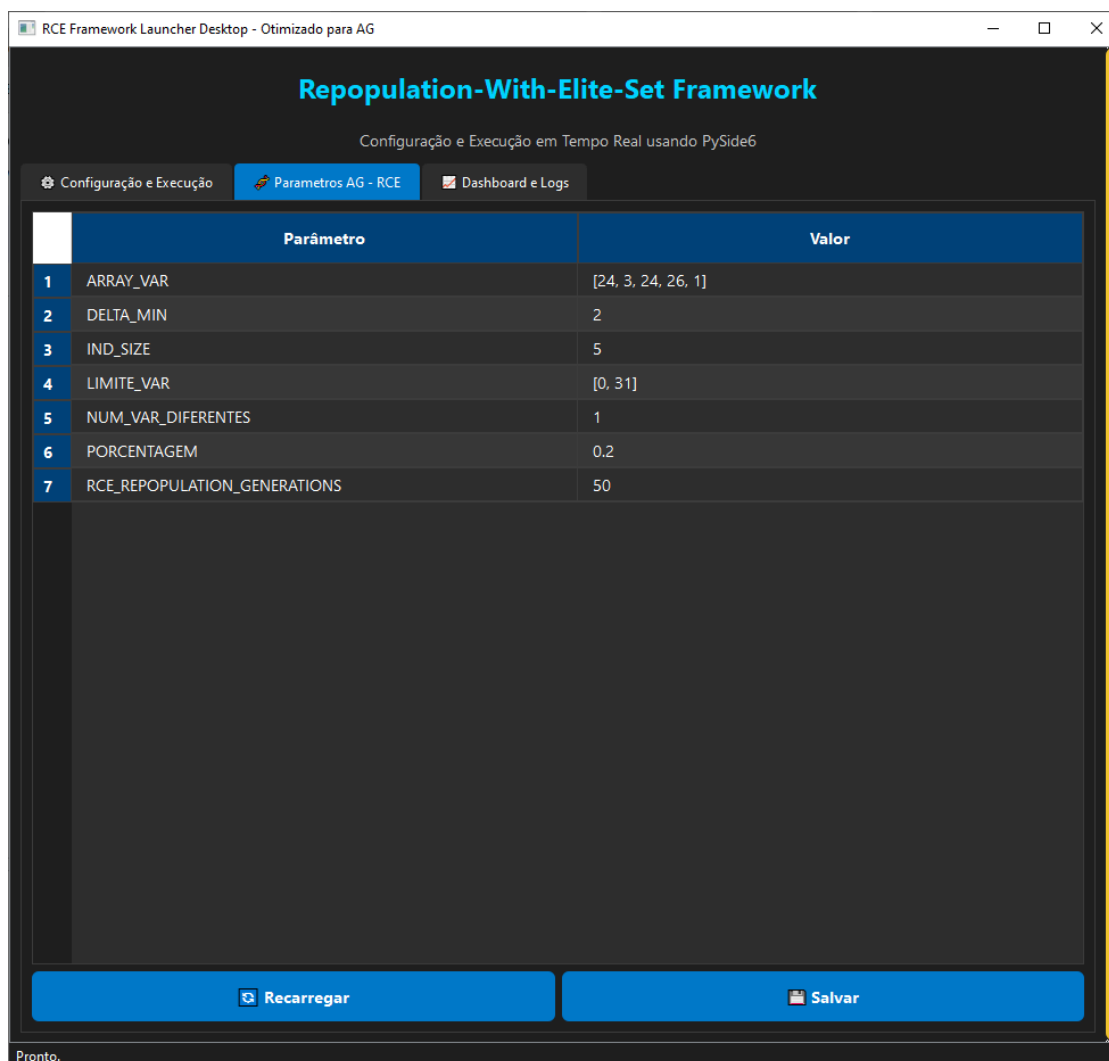
Figura 10: Tela do Launcher na aba “Configuração e Execução” com múltiplas configurações



Fonte: Autoria própria

Outros parâmetros de configuração do AG podem ser editados na aba “Parâmetros AG - RCE” do *Launcher*, ilustrada na Figura 11. Nesta aba é possível carregar, editar e salvar os parâmetros dos arquivos de configuração “options.json” e “params.json”.

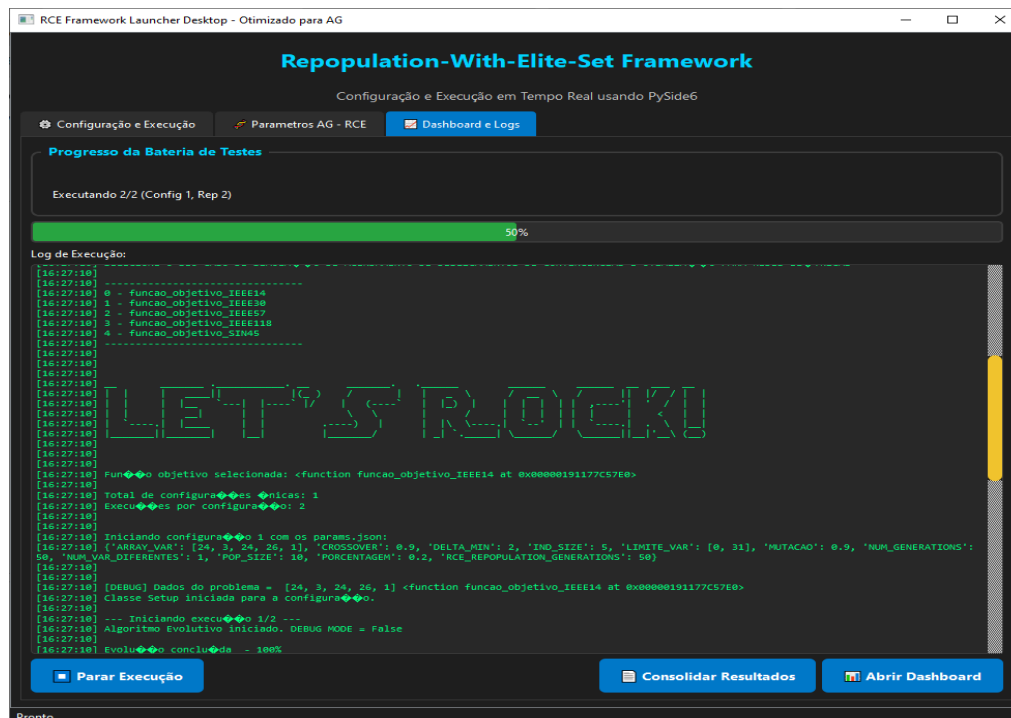
Figura 11: Tela do Launcher na aba “Parâmetros AG-RCE” com outras configurações do AG



Fonte: Autoria própria

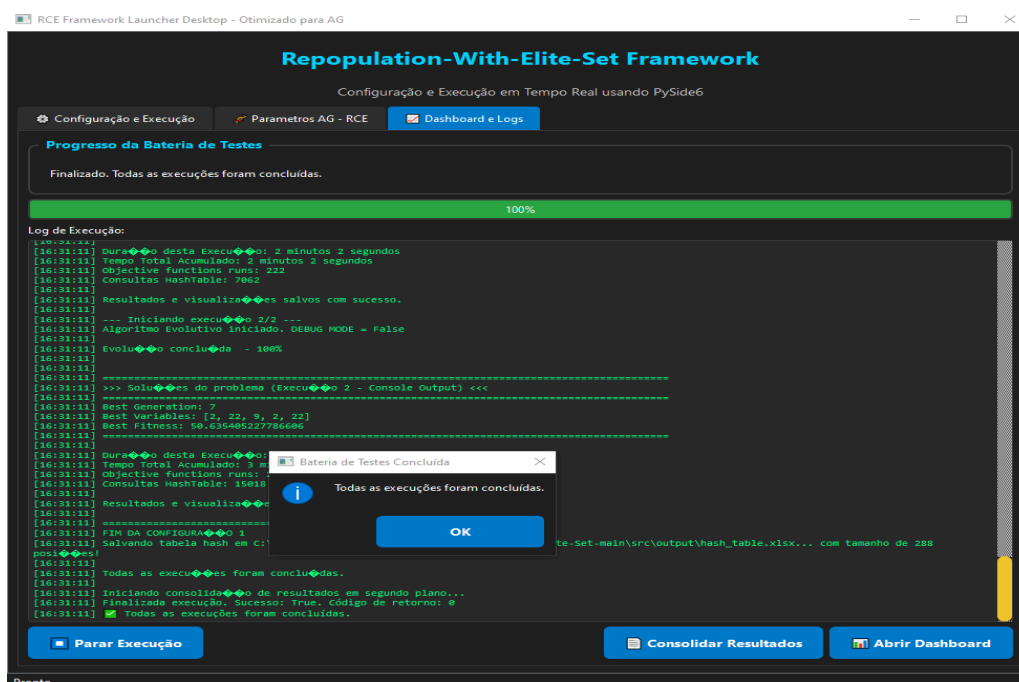
Depois de todos os parâmetros configurados nas duas abas anteriores, o usuário deve retornar à aba “Configuração e Execução” e pressionar o botão “Salvar e Executar”. Neste momento, a aba “Dashboard e Logs” é ativada para que o usuário verifique o terminal com o andamento das tarefas executadas durante a execução do *framework*. Na Figura 12, é possível ver um exemplo de uma execução com esta aba habilitada. É possível interromper a execução, consolidar resultados ou abrir o dashboard a partir desta aba. Na Figura 13, é possível visualizar um exemplo de tela mostrada ao final da execução de todas as execuções.

Figura 12: Tela do Launcher na aba “Dashboard e Logs” mostrando o andamento da execução



Fonte: Autoria própria

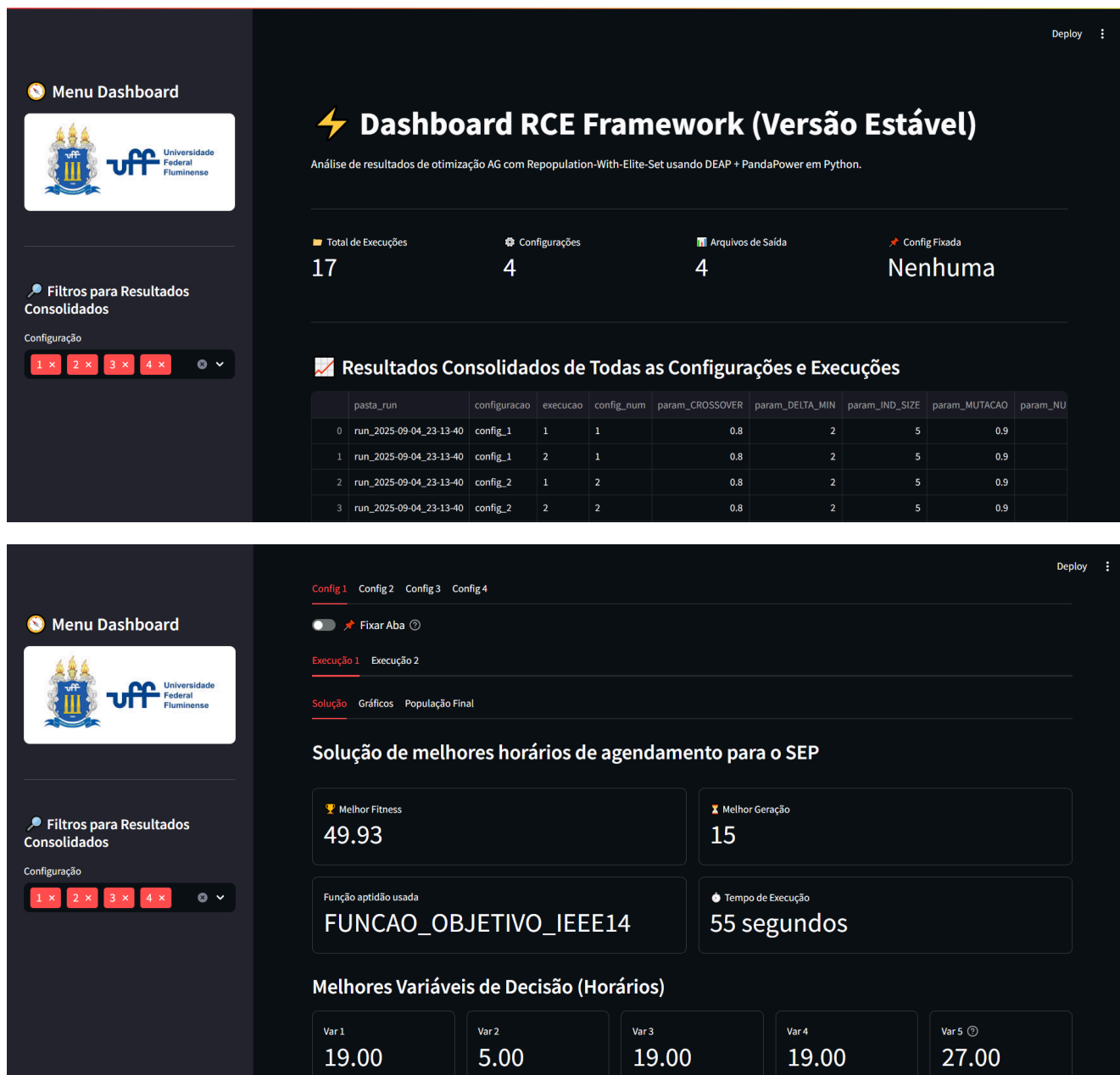
Figura 13: Tela do Launcher na aba “Dashboard e Logs” mostrando o final da execução



Fonte: Autoria própria

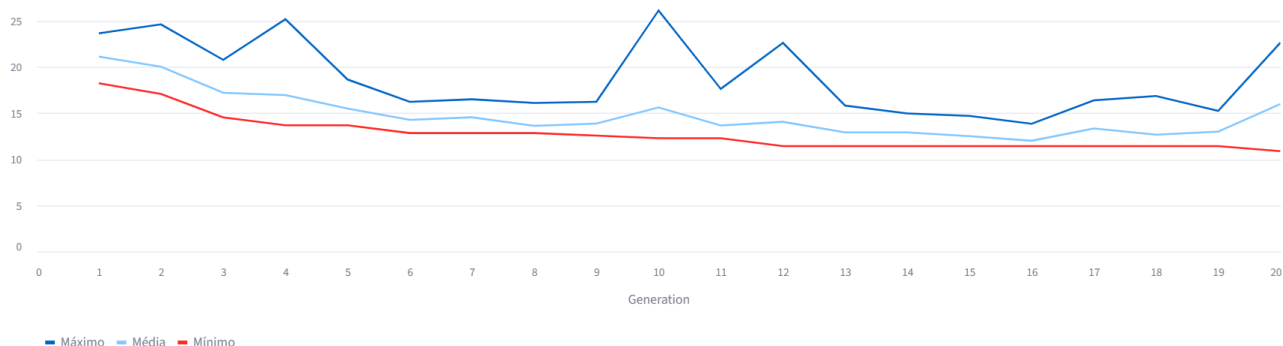
Após a finalização da execução, é possível acessar os dados consolidados através do *dashboard* que é aberto em uma nova página do *browser* padrão do sistema operacional para páginas *web*, onde a visualização pode ser feita através de tabelas exportáveis em planilhas Excel e gráficos. As opções de visualização da execução são divididas em 3 abas de resultados: Soluções, Gráficos e População final. Na Figura 14, é apresentada uma tela inicial deste *Dashboard*, na aba de Soluções, onde é possível navegar entre os dados obtidos da função objetivo em todas as simulações configuradas e executadas pelo *Launcher*. Na Figura 15, é apresentado o gráfico de linhas gerado por cada execução com o uso da estratégia RCE, onde é possível ver o comportamento da otimização e da minimização dos valores de *fitness* a cada geração. Na Tabela 1, é mostrado o resultado da população final realizada pelo *DEAP* em uma simulação de 20 gerações de população com 10 indivíduos e com uso da estratégia de RCE. É possível verificar a diversidade de soluções nos melhores horários de agendamento.

Figura 14: Telas do *Dashboard* para visualização dos resultados consolidados de todas as execuções



Fonte: Autoria própria

Figura 15: Gráfico de aptidão mínima, média e máxima a cada geração usando a estratégia RCE



Fonte: Autoria própria

Tabela 1: População final ao final da simulação

Variáveis de Decisão	Fitness	RCE	Diversidade
[5, 13, 2, 6, 14, 7, 5, 7, 4, 6]	12,27866011	HOF	69
[5, 13, 2, 6, 14, 7, 1, 7, 26, 6]	13,40642625	SIM	87
[5, 13, 2, 6, 14, 28, 5, 7, 6, 6]	13,68640538	SIM	92
[13, 13, 2, 6, 14, 7, 5, 7, 6, 6]	13,96638451	SIM	79
[20, 21, 8, 5, 16, 24, 8, 11, 8, 7]	17,39182639		128
[5, 9, 4, 13, 24, 25, 3, 20, 4, 12]	21,46014023		119
[9, 2, 17, 12, 11, 24, 19, 12, 16, 16]	24,49694378		138
[25, 7, 22, 29, 18, 18, 7, 18, 25, 10]	17,39182639		179
[23, 25, 11, 9, 12, 7, 7, 0, 26, 31]	18,63851037		151
[27, 14, 25, 16, 26, 31, 10, 26, 6, 16]	17,52300211		197

Fonte: Autoria própria

Para validar o uso da implementação do projeto, são apresentados nas seções seguintes os resultados de simulações realizadas em todos os sistemas teste analisados para o agendamento ótimo de intervenções em redes elétricas [1].

5.1) Simulações do problema de agendamento ótimo na rede IEEE 14

Nas simulações realizadas para a rede IEEE 14, observou-se que o algoritmo evolutivo foi capaz de identificar soluções de qualidade com eficiência computacional satisfatória. Considerando o conjunto de aproximadamente 387 execuções, distribuídas em quatro configurações distintas com 20 e 100 gerações, os resultados apresentaram o valor de *fitness* com uma variação entre 40,49 e 101,20 ao longo das execuções. Em uma das execuções representativas, a melhor solução foi obtida na geração 14, com variáveis [0, 7, 0, 0, 16] com o valor de *fitness* de 40,49. O tempo total de processamento para essa execução foi de 1 minuto e 14 segundos, confirmando a viabilidade da abordagem em termos de desempenho através do uso da tabela *hash*. A análise evidencia que a estratégia RCE contribuiu de forma consistente para a redução da função objetivo, enquanto a natureza estocástica do Algoritmo Genético proporcionou diversidade nas soluções, reforçando a robustez da metodologia proposta para o problema de agendamento de manutenções.

Na figura 16, é ilustrada uma captura de tela desenvolvida em *Streamlit* durante o projeto como solução para experimentação e pesquisa sobre recursos de visualização de dados. As simulações foram separadas por dias e por 4 configurações para cada função objetivo. O dashboard interativo em *Streamlit* possui as principais ferramentas da biblioteca *Pandas* para análise de dados e possui gráficos e filtros customizados pela sua base de dados.

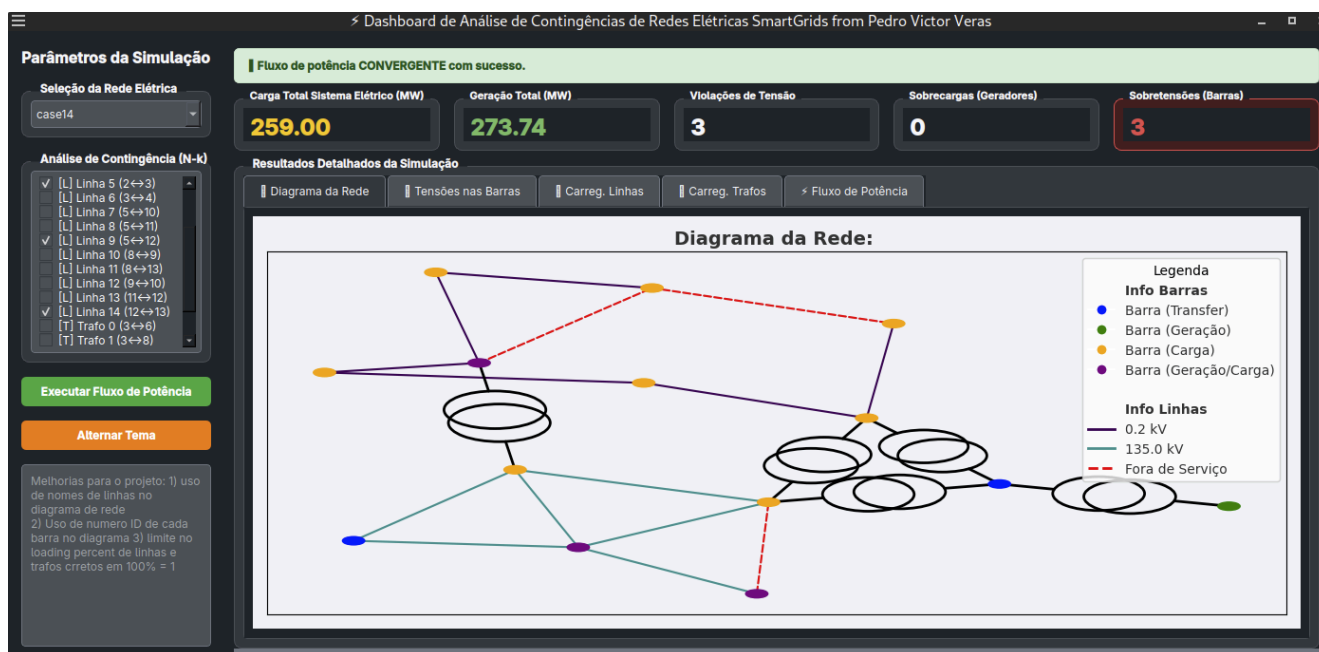
Figura 16: *Dashboard* interativo para explorar os resultados



Fonte: Autoria própria

Para verificação de cenários individuais de cada solução, foi desenvolvido um simulador que permite realizar o desligamento manual dos ramos (programados e em contingência) e executar o fluxo de potência. Na Figura 17 é possível verificar os desligamentos dos ramos de forma interativa em um aplicativo *desktop* em *PySide6* totalmente nativo em *Python*.

Figura 17: Representação de cenário com os ramos desligados na rede IEEE 14



Fonte: Autoria própria

No simulador é possível verificar as tensões nas barras e estimular os limites operacionais de 0,95pu e 1,05pu como é feito na operação do ONS [3]. Na Figura 18, é possível ver o *plot* do simulador dos comportamentos das tensões em cada barras.

Figura 18: Tensões nas barras para caso IEEE 14



Fonte: Autoria Própria

Na Figura 19, são apresentados os resultados do fluxo de potência ativa (MW) e reativa (MVar) para a rede IEEE 14. Observa-se que a potência ativa apresenta valores elevados em determinados ramos, ultrapassando 160 MW, enquanto em outros há quedas significativas, inclusive fluxos negativos, o que indica inversão no sentido da potência. Já a potência reativa apresenta comportamento oscilatório, variando aproximadamente entre -20 e $+12$ MVar, evidenciando a necessidade de compensação em pontos específicos da rede para manter a estabilidade de tensão. Esse resultado demonstra como a topologia da rede influencia diretamente a distribuição de cargas e reforça a importância da avaliação detalhada de fluxos ativos e reativos no processo de agendamento de manutenções. Ressalta-se ainda que o cálculo foi realizado com o método iterativo de *Newton-Raphson*, implementado no *PandaPower*, que dispõe de suporte nativo ao compilador *Numba*, possibilitando reduzir significativamente o tempo de execução das simulações.

Figura 19: Fluxo de potência Ativa / Reativa Caso IEEE14



Fonte: Autoria Própria

5.2) Simulações do problema de agendamento ótimo na rede IEEE 30

No caso da rede IEEE 30, foram realizadas simulações com 20 gerações, adotando-se este cenário como principal referência para avaliar a eficiência do mecanismo de consulta à tabela *hash*. O melhor valor de *fitness* obtido foi 40, resultado consistente com as execuções de teste.

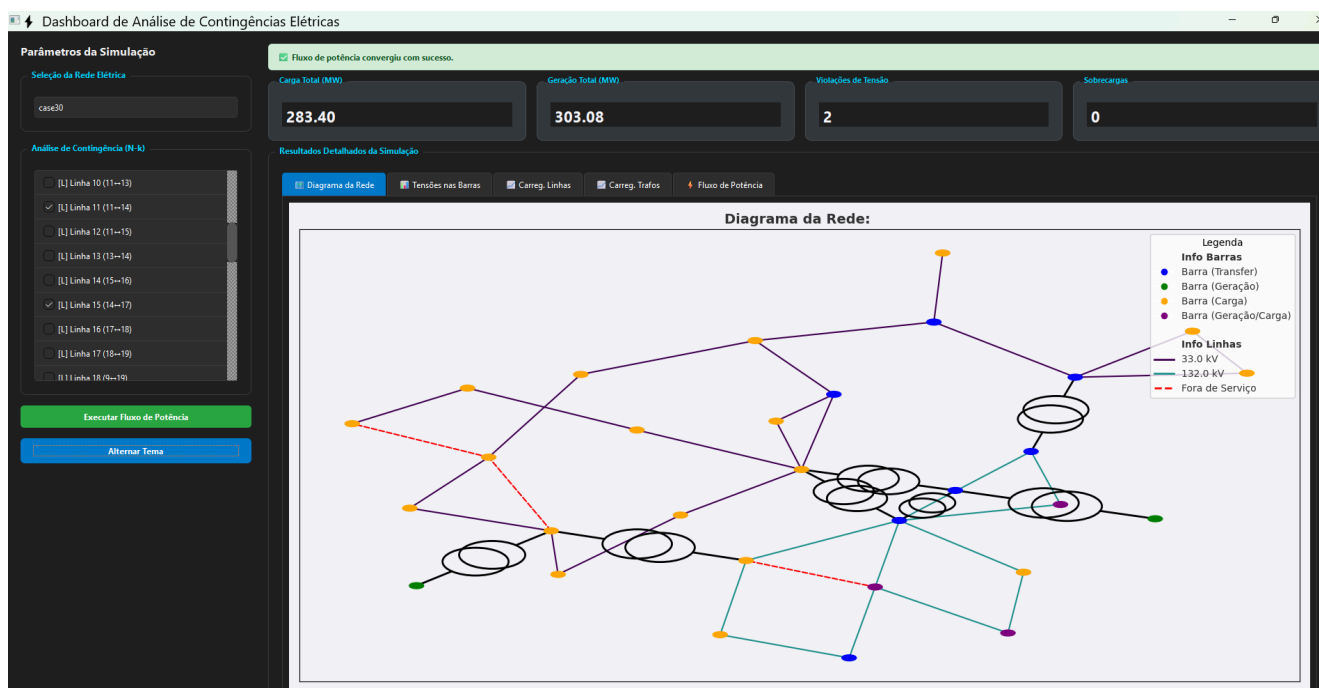
De acordo com o apresentado no trecho de Código 5, a classe *Setup e AlgEvolutivoRCE* faz uso da tabela *hash* com tamanho igual ao número total de cenários para melhor eficiência e performance no código. É possível calcular o número total de cenários usando uma função matemática como visto em [1].

$$tamanho_{hash} = num_{contingências} \times num_{carregamentos} \times 2^{num_{desligamentos}}$$

A tabela *hash* é gerenciada pelo objeto *Setup* que é passada como argumento para a função. Isso cria um acoplamento entre a função de *fitness* e a classe *Setup*. A tabela *hash* existe apenas na memória durante a execução de cada configuração do algoritmo. Se rodar o programa novamente, todos os cálculos são refeitos do zero e a tabela *hash* é criada com o tamanho do seu vetor pelo número de casos a serem avaliados. A tabela *hash* possui contadores do número de consultas e o número de vezes que foi calculado um cenário novo.

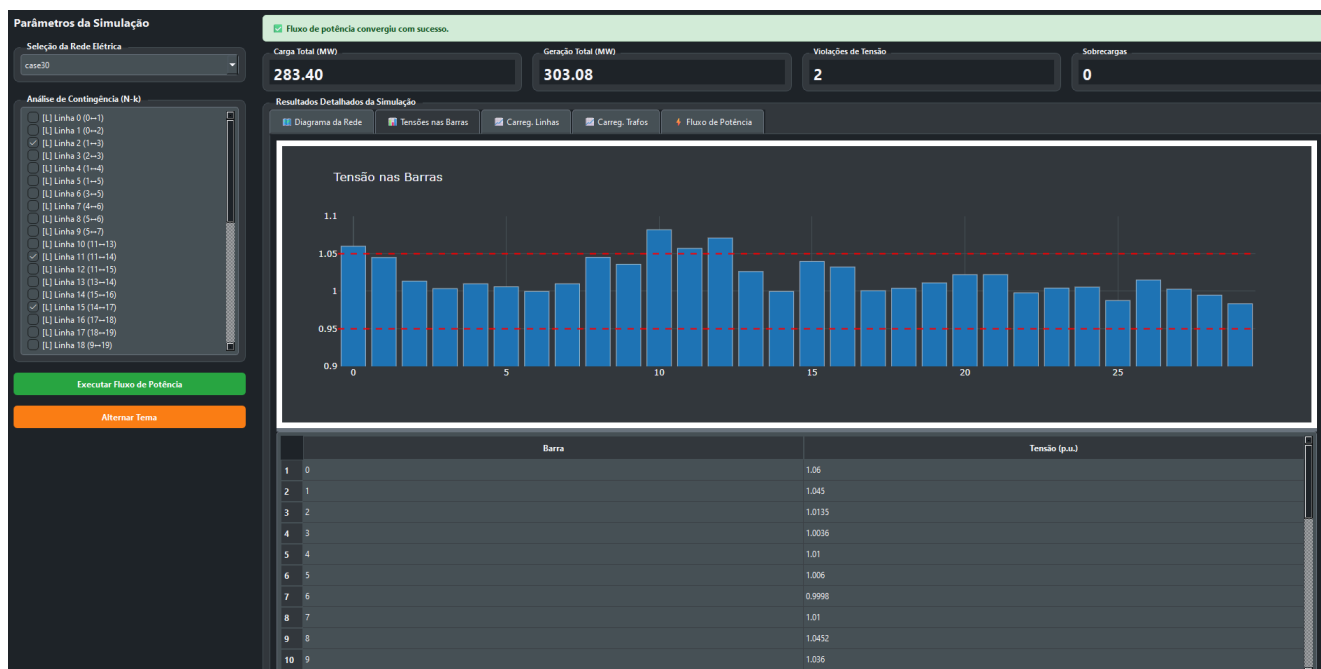
Através de uma consulta à tabela *hash* é possível saber os cenários já calculados para a rede elétrica desejada. Essa informação é armazenada no objeto *Setup* e atualizada pelo AG quando um novo cenário ocorre. Ao fazer consultas em memória para não precisar executar o fluxo de potência novamente, é possível reduzir o tempo de execução e melhorar a eficiência computacional. Por exemplo, para a rede IEEE 30, em um total de 9216 cenários possíveis, foi verificado o contador '*Objective functions runs*' igual a 3057. Esse foi o número de vezes que a otimização precisou realizar o cálculo de fluxo de potência de um cenário ainda não visitado. Para o mesmo caso, o contador '*Consultas HashTable*' registrou 55620, mostrando o número de vezes que o algoritmo já conhecia o valor de *fitness* do cenário visitado e apenas fez a consulta em memória para buscar rapidamente esse valor. Na Figura 20, é possível verificar os desligamentos dos ramos solicitados no diagrama da rede. Na Figura 21, é possível visualizar o comportamento das tensões em cada barra e, na Figura 22, é possível ver o funcionamento da otimização no comportamento da rede elétrica e com o ramo desligado, mostrando que é viável realizar a operação em SEP no horário agendado.

Figura 20: Representação de cenário com os ramos desligados na rede IEEE 30



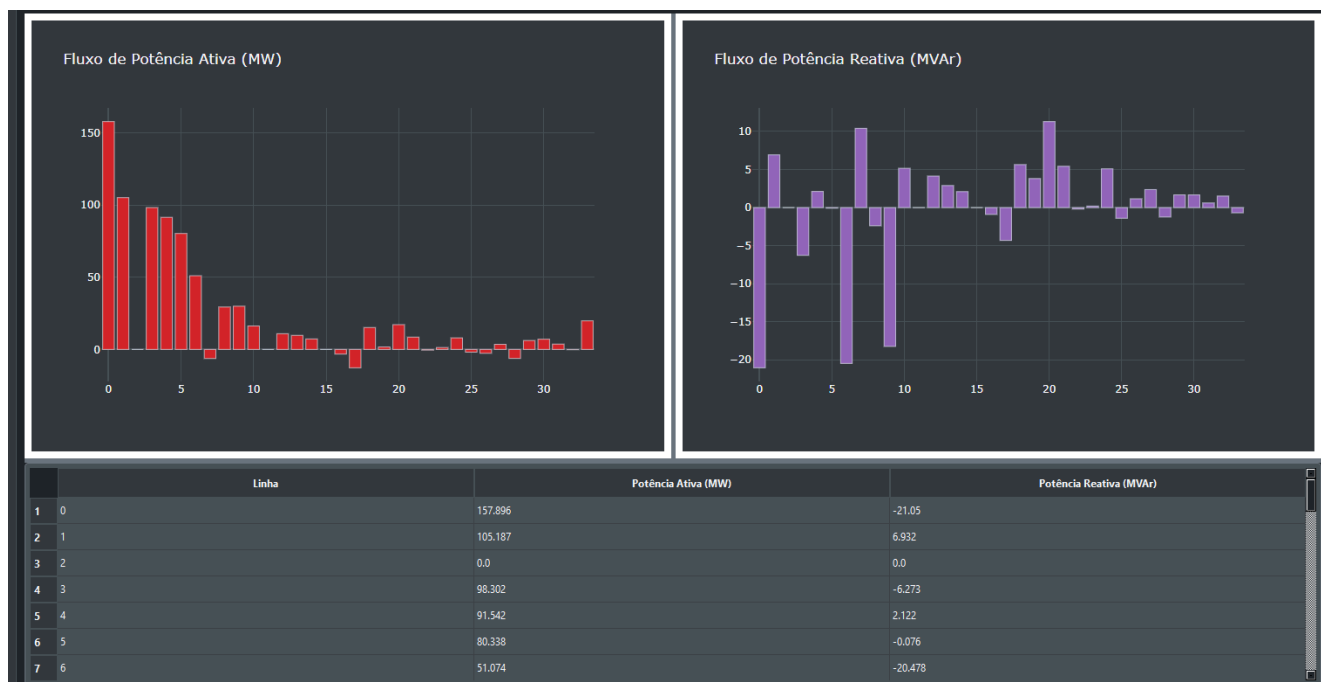
Fonte: Autoria Própria

Figura 21: Tensões nas barras para caso IEEE 30



Fonte: Autoria Própria

Figura 22: Fluxo de potência Ativa / Reativa Caso IEEE 30

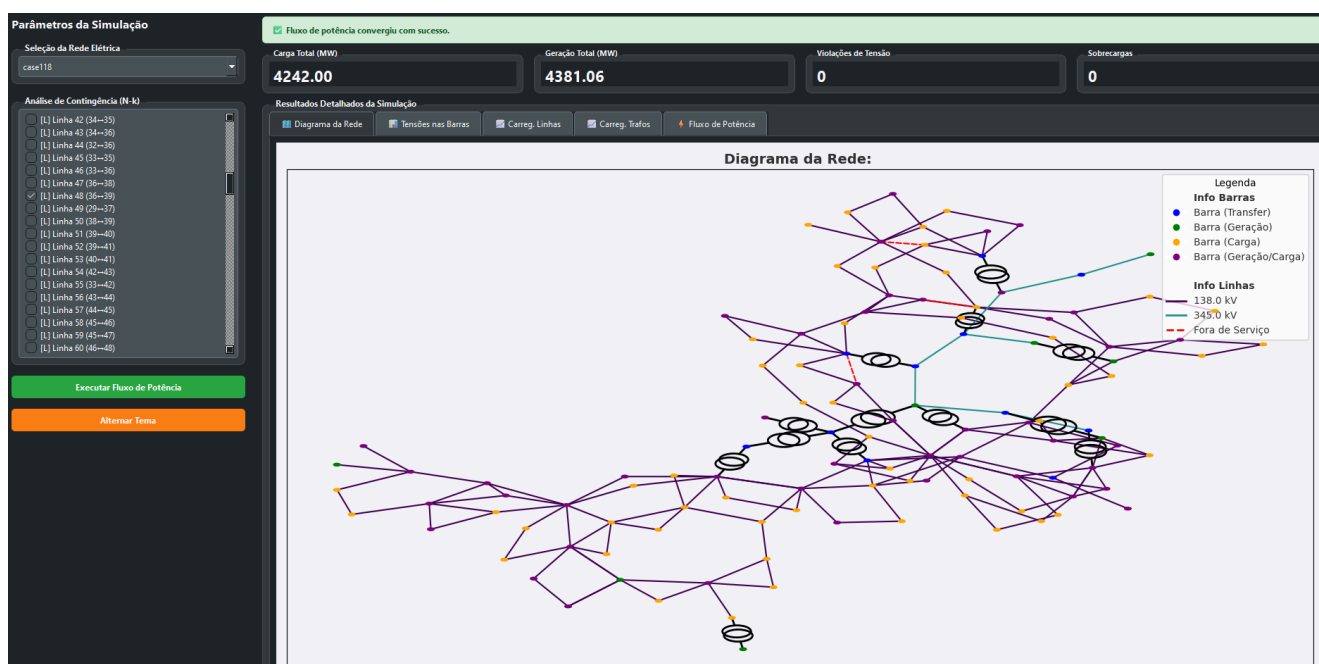


Fonte: Autoria Própria

5.3) Simulações do problema de agendamento ótimo na rede IEEE 118

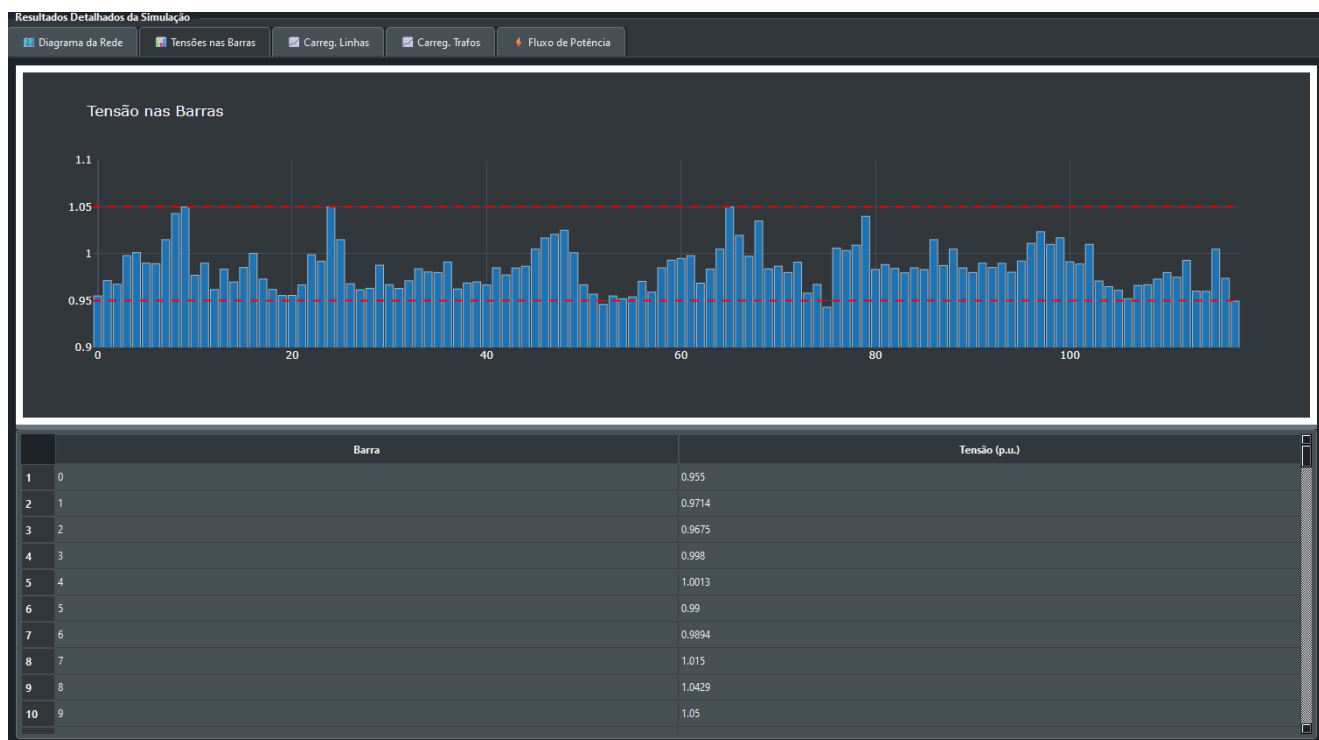
Para a rede IEEE 118, foram realizadas simulações com 20 gerações, buscando avaliar o comportamento do Algoritmo Genético em um sistema de maior porte. Os resultados indicaram valores de *fitness* consistentemente baixos, em média de valor 11, na maior parte das execuções, evidenciando a adequação do método para o caso. Em determinados cenários, o processo apresentou tanto casos convergentes quanto divergentes, dependendo da configuração avaliada, o que reforça a necessidade de ajustes finos nos parâmetros do modelo para garantir estabilidade. A Figura 23 ilustra a análise de contingência com os ramos selecionados pelo AG, evidenciando a capacidade do algoritmo em identificar pontos críticos da rede. A Figura 24 apresenta as tensões nas barras para o caso IEEE 118, permitindo verificar a manutenção dos níveis dentro de limites aceitáveis, enquanto a Figura 25 mostra o comportamento dos fluxos de potência ativa e reativa, destacando a consistência dos resultados obtidos. A melhor simulação reportada apresentou *fitness* igual a 6,93, associado ao vetor de horários de agendamento [3, 25, 4, 4, 4, 3, 31, 3, 4].

Figura 23: Representação de cenário com os ramos desligados na rede IEEE 118



Fonte: Autoria Própria

Figura 24: Tensões nas barras para caso IEEE 118



Fonte: Autoria Própria

Figura 25: Fluxo de potência Ativa / Reativa Caso IEEE 118

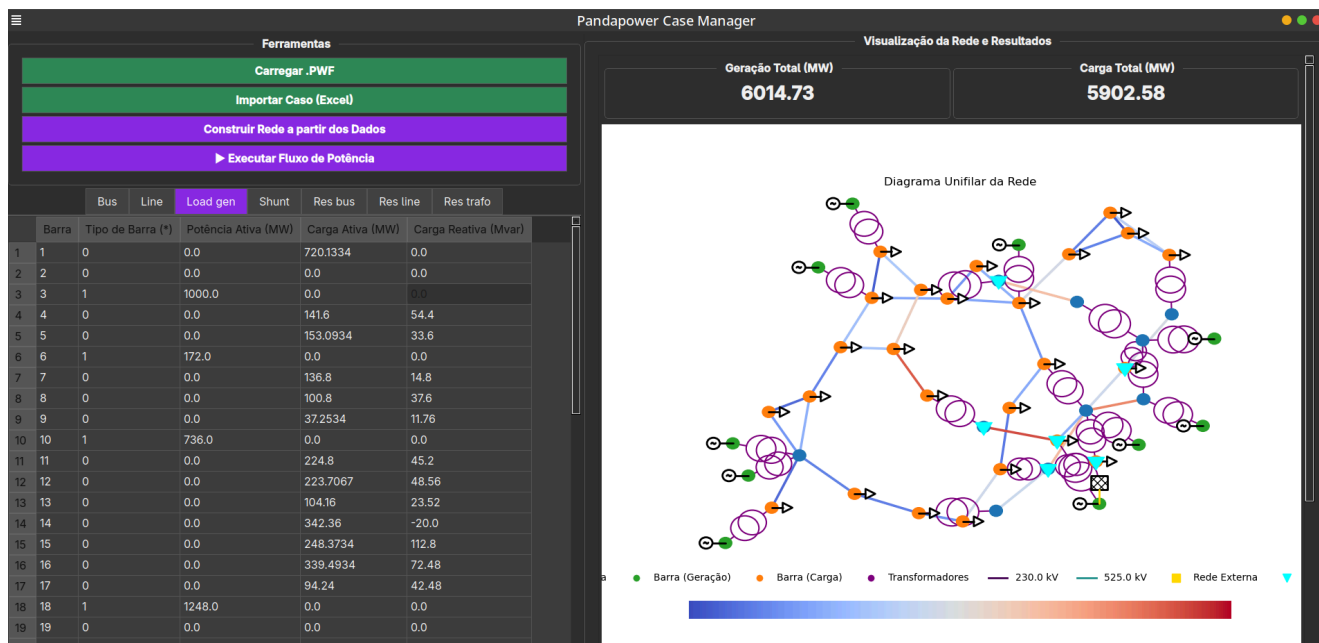


Fonte: Autoria Própria

5.4) Simulações no SIN 45 como exemplo de controle de casos

Na rede SIN 45, foi implementado e validado um novo simulador exclusivo para essa rede, em *PySide6*, integrado à função objetivo, permitindo a aplicação direta do AG em uma topologia mais próxima da realidade do SIN. Os resultados preliminares confirmaram a viabilidade da metodologia, ainda que alguns cenários tenham apresentado casos divergentes. Em uma das simulações com 100 gerações, registrou-se fitness de 1041 e tempo de execução de aproximadamente 27 minutos, o que demonstra o custo computacional associado a casos com mais cenários sem atendimento à demanda. Além disso, todos os experimentos geraram *plots* detalhados para inspeção manual no *dashboard*, incluindo diagramas da rede, análises de contingências e gráficos dinâmicos de tensões nas barras, carregamentos em linhas e transformadores, bem como fluxos de potência ativa e reativa. Tais resultados indicam que, com ajustes adequados de parametrização e estratégias de controle de convergência, o *framework* é aplicável ao sistema teste, mantendo potencial de utilização prática em estudos de operação e planejamento. Na Figura 26, é ilustrada uma captura de tela do novo simulador onde é possível utilizar os dados de entrada da rede em arquivos de formato *.xlsx* e *.PWF*, usado em outros simuladores do mercado, como o AnaREDE. No simulador, é possível gerar um relatório completo da rede tanto em arquivos *.PDF* quanto *.HTML* sobre as tensões nas barras, carregamento das linhas e resultado detalhado sobre o fluxo de potência.

Figura 26: Simulador SIN 45



Fonte: Autoria própria

Na seção seguinte, são apresentados roteiros passo a passo para utilização do *framework* desenvolvido neste projeto a partir de seu repositório no *Github*.

5.5) Passo a passo para utilização do *framework* pelo *Github*

1. Estrutura do Projeto: A estrutura principal do projeto foi organizada da seguinte forma:

- Contém o script de execução do algoritmo em `src/run.py`, incluindo a lógica de *Setup*, o algoritmo evolutivo e as funções de *fitness* adaptadas para o fluxo de potência de cada rede.
- Diretório onde todos os resultados das execuções (gráficos, *logs* e planilhas) são salvos na pasta `output/`
- Dentro da pasta *src/* é possível encontrar as pastas de cada classe criada que podem funcionar independente uma da outra.

2. Configuração do Ambiente: Antes de executar o framework, certifique-se de que todos os pré-requisitos estão instalados.

Pré-requisitos : Python 3.8 ou superior

Para instalar todas as bibliotecas necessárias, execute o seguinte comando no terminal, a partir da raiz do projeto:

```
Shell  
pip install -r requirements.txt
```

Se você usar Windows, pode usar o arquivo *instalador.bat*

Para usuários linux, se você tiver o *Wine* instalado, é possível também executar o instalador

```
Shell  
  
chmod 777 instalador.bat  
  
./instalador.bat
```

3. Executando o Framework

Existem duas maneiras principais de executar o framework, sendo usando interface gráfica ou editando os parâmetros de configuração AG em *.json* e rodar o script *run.py* no terminal.

Método 1: Usando o Launcher Gráfico (Recomendado)

O `launcher.py` fornece uma interface em *PySide6* para configurar e executar os experimentos.

1. Abra o terminal na raiz do projeto.
2. Execute o launcher com o seguinte comando:

```
Shell  
python launcher.py
```

3. Na aba de Configuração e Execução:

- Pode-se configurar o arquivo `params.json` em tela, salvando o arquivo automaticamente.
- É necessário que o `IND_SIZE` e `ARRAY_VAR` devam ter a mesma dimensão
- Defina o número de execuções por configuração.
- Ajuste os parâmetros do algoritmo genético (Mutação, Crossover, etc.) usando os modos "Fixo" ou "Variável" em até 4 valores distintos.
- É feito o cálculo do número de configurações e total de execuções da simulação
- Clique em Salvar e Executar para rodar o script `run.py` em *subprocess*

4. Acompanhe a execução na aba "Dashboard e Logs".

- Ao clicar o botão “abrir dashboard”, no mesmo terminal é inicializado o *Streamlit* e abre no seu navegador principal por padrão no `localhost:8501`, os resultados do *framework* em tempo real
- É possível parar a simulação e ajustar o *Launcher* novamente
- A cada finalização do conjunto de execuções pré-definidas pelo usuário, é acionado o script para consolidar os resultados em um arquivo único em `.xlsx`, essa funcionalidade pode ser usada dentro do *Launcher* para consolidar resultados.

Método 2: Execução Direta via Script

Para usuários mais acostumados na área de programação, é possível executar o framework diretamente via terminal.

1. Configure os parâmetros nos arquivos `src/params.json` (parâmetros AG) e `src/options.json` (parâmetros variáveis e de execução).

2. Execute o script principal a partir da raiz do projeto:

```
Python  
python src/run.py
```

4. Visualizando os Resultados

Após a execução, os resultados são salvos na pasta `output/`. Para uma visualização interativa, use o Dashboard *Streamlit*.

1. Navegue até a pasta do Dashboard:

```
Python  
cd src/DashboardApp
```

2. Inicie o aplicativo *Streamlit*:

```
Python  
  
streamlit run dashboard_RCE_APP.py
```

3. Acesse o dashboard no seu navegador, geralmente no endereço <http://localhost:8501>.

Para alterar suas variáveis de decisão é necessário alterar o parâmetro `ARRAY_VAR` dentro do `params.json` tanto pelo seu editor de código ou pelo *Launcher*. Para alterar a função objetivo, é recomendável criar um script em `src/utils/fitness_functions/` com essas importações necessárias.

Com a função objetivo criada, realize a importação no arquivo `run.py` e passa o `params.json`, a função objetivo e o tamanho da tabela hash como argumentos para instanciar o objeto *Setup* antes de usar o método *AlgEvolutivoRCE.run()*.

6) CONCLUSÕES

Este trabalho apresentou um *framework* integrado, em *Python*, que combina algoritmos evolutivos (implementados com *DEAP*) e simulações de fluxo de potência (usando *Pandapower*) para o problema de agendamento de intervenções em sistemas elétricos de potência.

Os objetivos propostos para este estudo foram alcançados de forma satisfatória onde foi implementado os casos de uso real de agendamento de manutenção em redes elétricas em [1] utilizando *Pandapower* e *Python*. O algoritmo desenvolvido permite identificar e avaliar todos os cenários de interesse em um agendamento, conforme a função objetivo descrita em [1]. Na etapa final deste projeto, esta função, desenvolvida para 5 sistemas teste, foi integrada ao *framework* desenvolvido em [6]. Foram realizados testes para validar o uso da implementação final, realizando simulações com diversas execuções para cada configuração única obtida com a variação de diversos parâmetros. A adoção de uma tabela *hash* para armazenar resultados de cenários já calculados proporcionou redução substancial do esforço computacional, viabilizando a execução de grandes baterias de testes.

A metodologia e a ferramenta desenvolvidas podem ser úteis em diversas aplicações no contexto da operação de sistemas elétricos. Esta implementação pode auxiliar no planejamento da manutenção preventiva, permitindo agendar intervenções de forma otimizada. A implementação da metodologia proposta foi integrada a um *frontend* em *Python*, criando uma ferramenta de *software* mais completa e amigável para utilização em ambiente de produção. Esta ferramenta está disponível em repositório público com licença para uso livre e pode ser utilizada por engenheiros e técnicos da área de sistemas de potência para o planejamento e otimização da manutenção de redes elétricas.

Como sugestões para trabalhos futuros, podem ser propostos:

- Expandir os testes para redes de maior porte e para cenários realistas de mercado e despacho usando algoritmos genéticos.
- Estudar a aplicação em redes de despacho ou outras formas de geração.
- Explorar outras estratégias de diversificação e meta-heurísticas híbridas, além do AG, para comparar desempenho e custo computacional.
- Integração com arquivos de configuração da rede elétrica de simuladores consolidados no mercado, como AnaREDE e *Organon*.

7) REFERÊNCIAS BIBLIOGRÁFICAS

[1] ZANGHI, R. “Meta-heurísticas aplicadas ao Agendamento de Intervenções em Redes Elétricas”, Tese de Doutorado, Instituto de Computação, Universidade Federal Fluminense, [pdf]. Disponível em: <http://www.ic.uff.br/PosGraduacao/frontendtesdissertacoes/download.php?id=741.pdf&tipo=trabalho>, [Acessado em novembro de 2024], 2016.

[2] SILVA, L. G. “Agendamento de intervenções em redes elétricas considerando a otimização da segurança sistêmica”, Dissertação de mestrado, Engenharia Elétrica e de Telecomunicações da Universidade Federal Fluminense, 2022.

[3] ONS (Operador Nacional do Sistema Elétrico)., Procedimentos de Rede, Submódulo 10.14, rev 2016.12 “Requisitos operacionais para os centros de operação, subestações e usinas da rede de operação”, Rio de Janeiro: ONS, 2016.

[4] THURNER, L. et al., "Pandapower—An Open-Source Python Tool for Convenient Modeling, Analysis, and Optimization of Electric Power Systems," in IEEE Transactions on Power Systems, vol. 33, no. 6, pp. 6510-6521, Nov. 2018, doi: 10.1109/TPWRS.2018.2829021.

[5] RIVERA, R.; ESPOSITO, A.S.; TEIXEIRA, I. Redes elétricas inteligentes (smart grid): oportunidade para adensamento produtivo e tecnológico local. Disponível em: https://web.bndes.gov.br/bib/jspui/bitstream/1408/2927/1/RB%2040%20Redes%20el%C3%A9tricas%20inteligentes_P.pdf

[6] VERAS, P. V. R; ROCHA, D. M F; ZANGHI, R. Estratégias de diversificação em meta-heurísticas aplicadas a problemas de otimização em engenharia elétrica, Revista PIBIC UFF : Anais do Seminário 2024, 2024. ISSN 2317-4560

[7] ZANGHI, R.; SOUZA, J. C. S. de; FILHO, M. B. do C. Estratégias de diversificação para otimização da programação de intervenções em redes elétricas. Anais do 12o Congresso Brasileiro de Inteligência Computacional, ABRICOM, outubro 2015. Disponível em: <<https://doi.org/10.21528%2Fcbic2015-034>>.

[8] GAMMA, R; HELM, R; JOHNSON, R; VLISSIDES, J. “Design Patterns: Elements of Reusable Object-Oriented Software” 1994

8) AUTO-AVALIAÇÃO DO ALUNO

Este trabalho demonstrou um profundo e extenso estudo sobre Sistemas Elétricos de Potência, especificamente no Agendamento de Intervenções em Redes Elétricas. Foi realizado um levantamento bibliográfico abrangente, incluindo teses de referência, e consolidado o entendimento de conceitos complexos como fluxo de potência, análise de contingências e diferentes cenários operacionais. Minhas habilidades em linguagens de programação, especificamente, em *Python*, foram aprimoradas no desenvolvimento de um *framework* completo, abrangendo tanto o *backend* quanto o *frontend* dentro de um projeto.. No *backend*, fui responsável pela orquestração de algoritmos complexos do *DEAP* e *PandaPower* utilizando de princípios de *clean code* e *Design Patterns*, assegurando modularidade, manutenção e escalabilidade. Essa abordagem me trouxe mais maturidade como um desenvolvedor . No *frontend*, desenvolvi um *launcher* de *desktop* em *PySide6* para configuração intuitiva e um *dashboard* web interativo em *Streamlit* para visualização de resultados, garantindo uma interface acessível e profissional. O projeto seguiu, onde desenvolvi um senso crítico apurado para a criação de um *software* profissional que abstrai a complexidade técnica para o usuário leigo. A criação de um simulador tanto para os casos de uso de IEEE e do SIN 45 me trouxe discernimento sobre a criação de *softwares* que podem ser utilizados no mercado para análise de Redes Elétricas, principalmente para o SIN, usando uma tecnologia de código aberto, onde isso abre portas para novas possibilidades e trabalhos futuros. O *launcher* e o *dashboard* foram projetados para permitir que engenheiros e técnicos realizem o planejamento e a otimização de manutenção de redes elétricas sem a necessidade de conhecimento aprofundado em programação. A interface permite a configuração simples de parâmetros e a visualização clara de dados operacionais, como tensões e fluxos de potência, seguindo padrões de operação como os da ONS.

Este projeto representa apenas um marco inicial na minha capacidade de impulsionar a inovação na Engenharia Elétrica através da tecnologia. Utilizando exclusivamente a linguagem *Python*, utilizando vários tipos de *frameworks* diferentes. Minha visão se estende à exploração de soluções ainda mais avançadas para *Smart Grids* e *IoT*, vislumbrando um futuro onde a Engenharia Elétrica se beneficia plenamente das capacidades da programação e do avanço computacional com a chegada da Inteligência Artificial Estreita.

Apesar de desafios pessoais significativos, emocionais e psicológicos, a conclusão deste trabalho foi possível graças ao apoio de professores, colegas e tratamento psicoterapêutico com Terapia Cognitivo-Comportamental. O ambiente acadêmico da Universidade Federal Fluminense e a orientação do professor Rainer Zanghi foram importantes para manter a motivação e o foco, transformando os obstáculos em uma jornada de crescimento e desenvolvimento pessoal, guiado e orientado pela Espiritualidade e com um aprendizado contínuo que resultou na entrega do projeto de um simulador com tecnologia de ponta.